

Generative AI

Made with Goodnotes



Generative AI became popular when OpenAI launched ChatGPT.

Instruction (prompt) → Generative AI model → Text response

Other Types of AI

- → Fraud detection
- → Recommender systems
- → Image recognition
- → Spam detection

} Machine learning models

Traditional AI



Recognise the cat







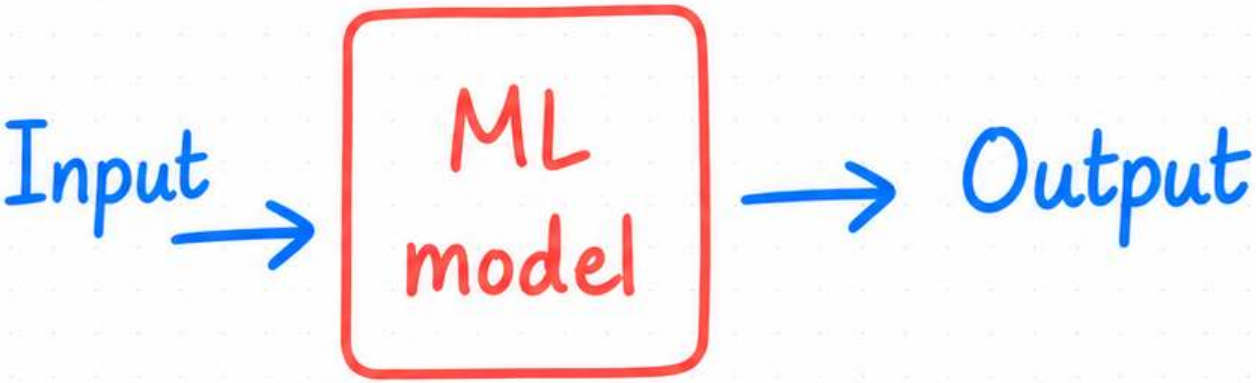




Supervised Learning

(learning with a teacher)

Humans label all the data



- Expensive, slow & limited
- AI can learn only as much as humans can label

Self-supervised Learning

(without a teacher)



Teacher labels the cat



Reads book independently

✓ No labels needed

✓ AI can learn as much as humans have ever recorded

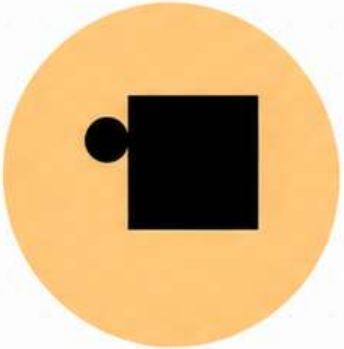
Gen AI



Predicts the missing piece

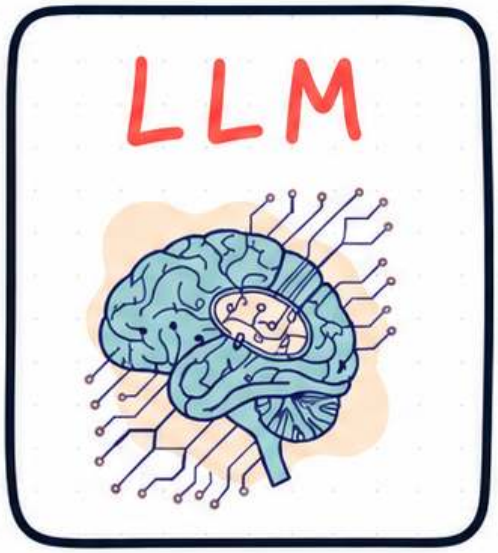
✓ The cat sat on the ___

✓ Paris is the capital of ___



← Predicts the missing part of the picture

Input
(prompt)



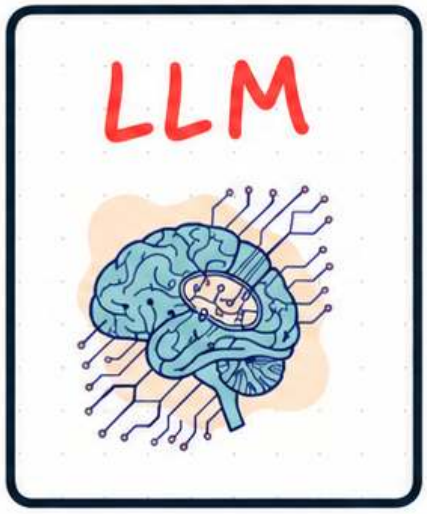
Output
(Text / Audio / Video)

Large → huge training data +
huge parameters (weights)

Language → human language

Model → mathematical system that
makes predictions

Given the previous words, what's the next most likely word to occur?



Capital of France is ___ → Paris

Input → LLM → Output

Broken into tokens

One token at a time

Tokens = units of text

Once | upon | a | time

un | believ | able

Input

↓

Broken into tokens

→

LLM



→

Output

↓

One token at a time

Input size is limited by the model's

CONTEXT WINDOW

Context

The model's working memory

Everything that goes into the model

Your prompt →

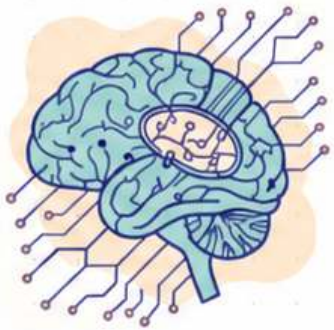
Entire conversation →

Uploaded files →

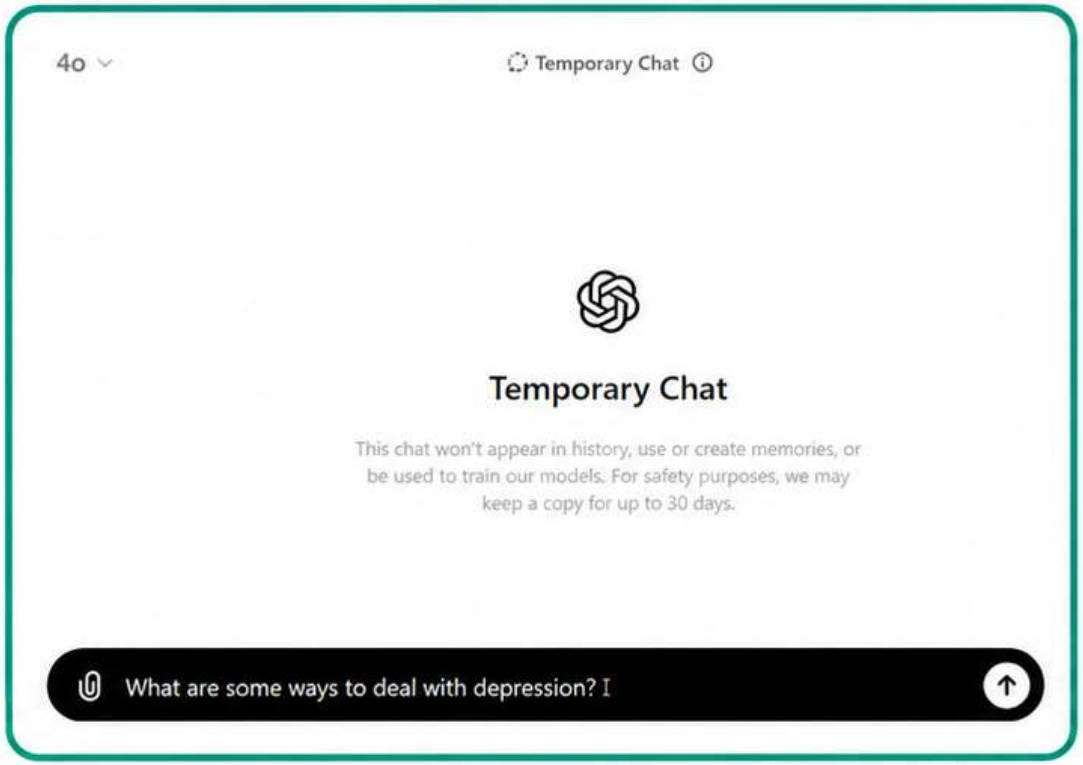
Instructions →

Model's own previous answers →

LLM



How conversation context grows



Each new message includes tokens from previous messages.



Models have no memory between conversations.

→ A new conversation clears the context.

Why manage context?

- ✓  The model forgets instructions
- ✓ Output quality degrades
- ✓  Higher cost — you consume more tokens
- ✓  Slower responses



Pricing

$ = Input tokens + Output tokens

↓ ↓

Context (prompt + instructions + ...)

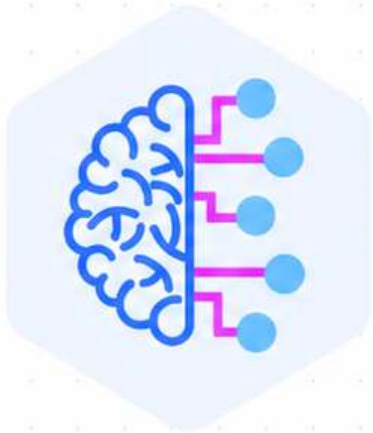
Model response

$ 30 + $70 = $100

Reducing Tokens

Fewer tokens ≈ Lower cost

1. Write concise prompts
2. Limit output length
3. Compress conversation history
   or start a new session



Small models
for trivial tasks



Skills
for repeated tasks



Scripts
for deterministic tasks





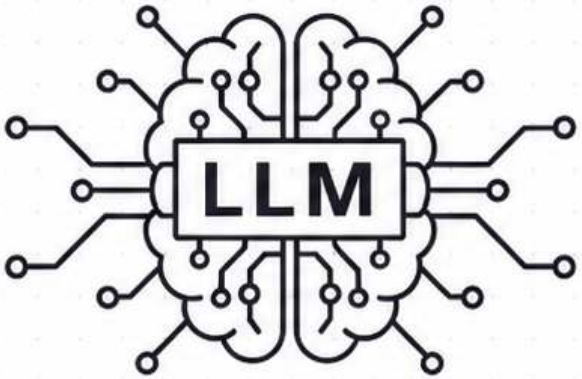


- Opus (Reasoning)
- Sonnet (Balanced)
- Haiku (Fast)

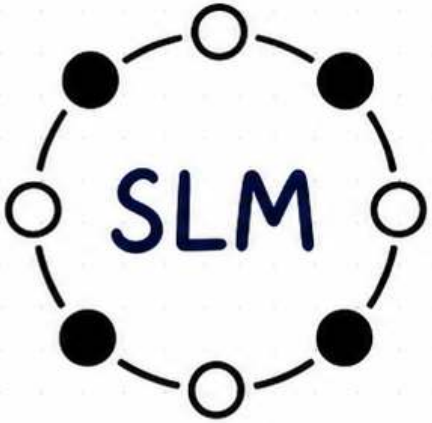
- GPT-5.5
- GPT-4o
- GPT-4.1
- GPT-4o mini

- Ultra
- Pro
- Flash
- Nano

LLMs vs. SLMs



- ✓ Better output
- ✓ Large context
- − Slow
- − Expensive



- − Good for trivial tasks
- − Fast
- − Cheap
- − May hallucinate

❓ Use Small Models



Draft emails



Fetch Jira ticket

Use Large Models



Architectural design



Complex coding

Plan with a large model
Implement with balanced models



Opus for planning



Sonnet for coding



Haiku / GPT-4o mini



Opus for review

Create tickets

Create MRs

LLM Trade-offs

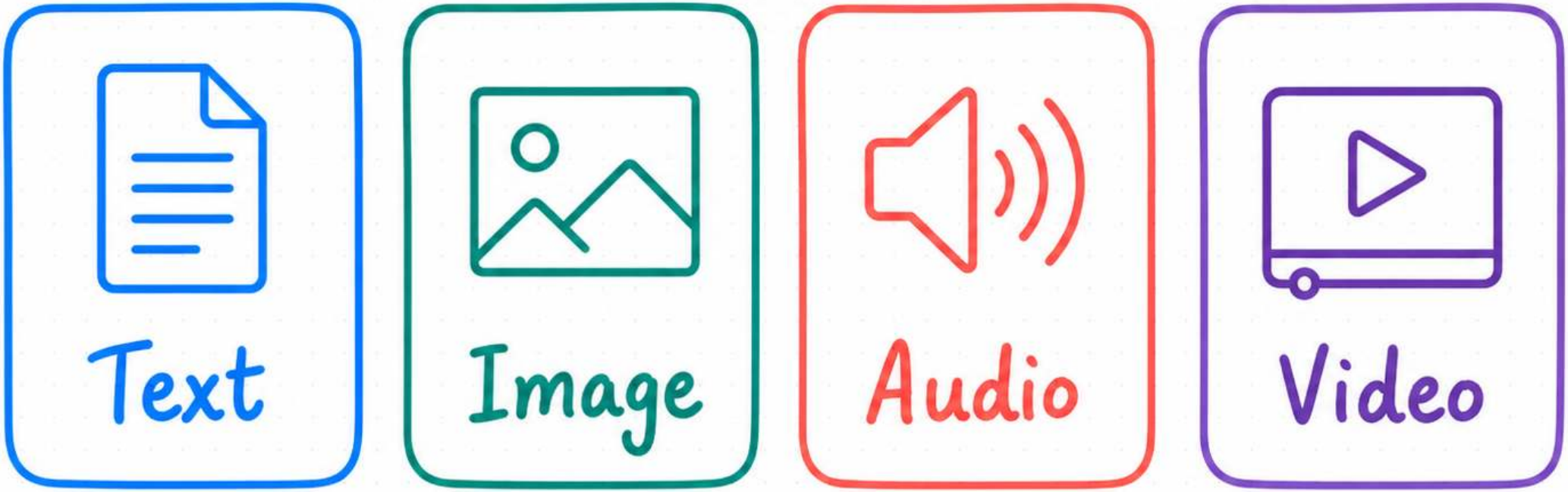
→ Non-deterministic

- Best salons in Amsterdam

→ Can hallucinate

Multimodals

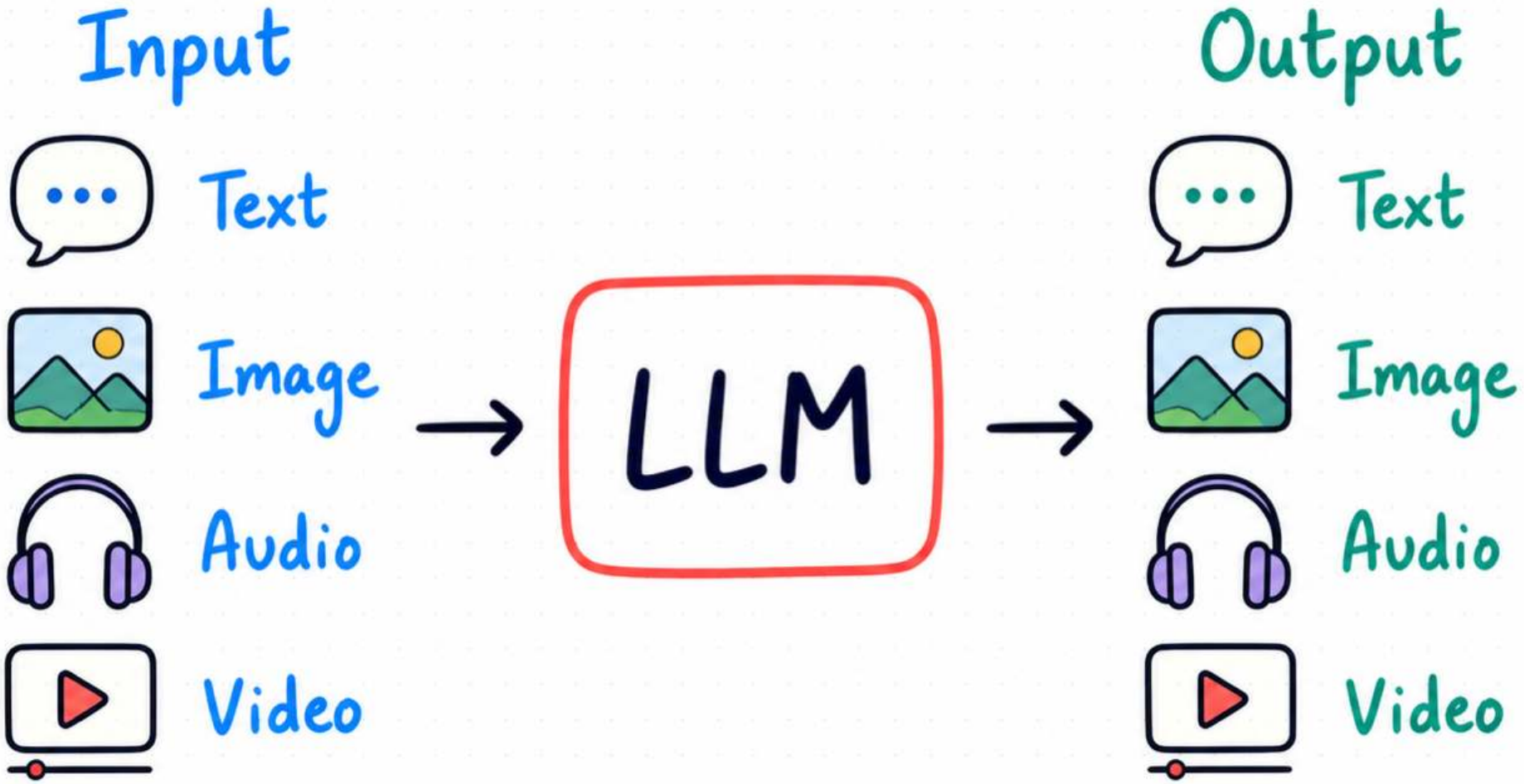
Modalities



Traditional LLMs



Advanced LLMs

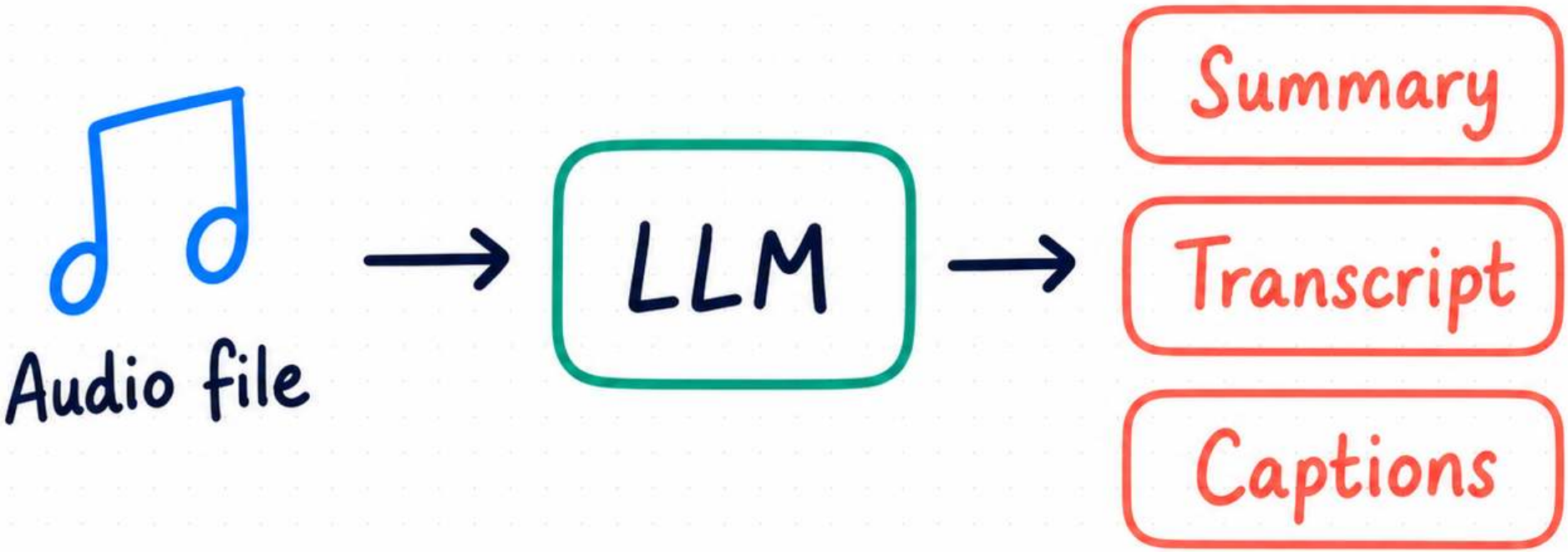


Why?

Because most of the data in the world is not just text.



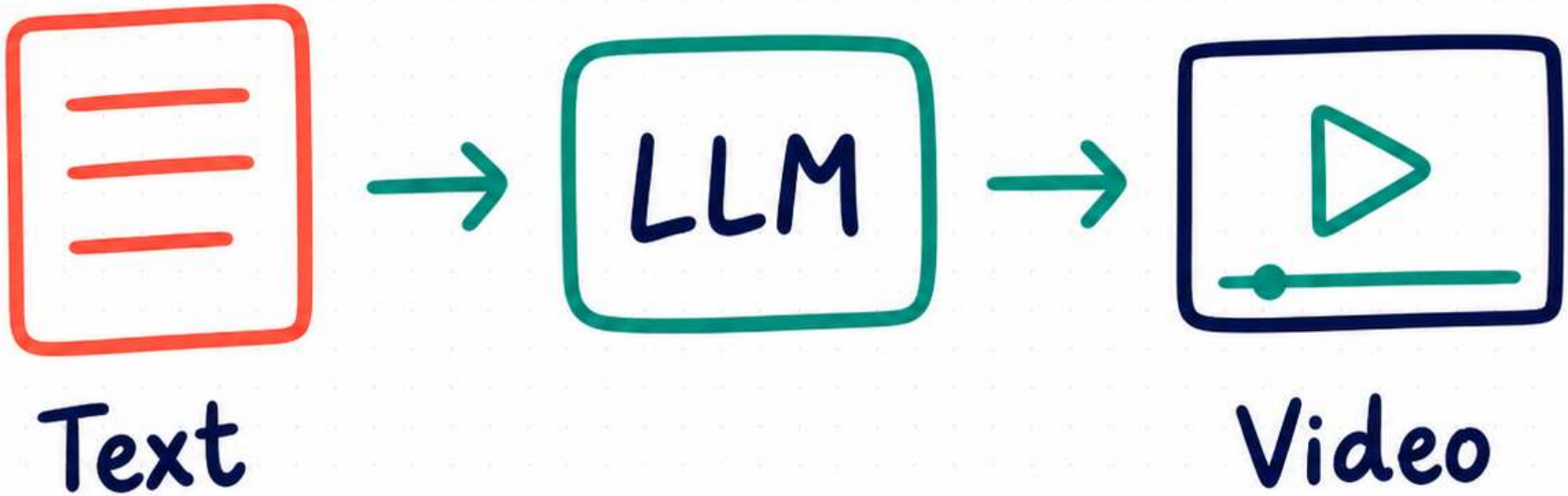
Use Cases



Resolve an Error



Generate a Video



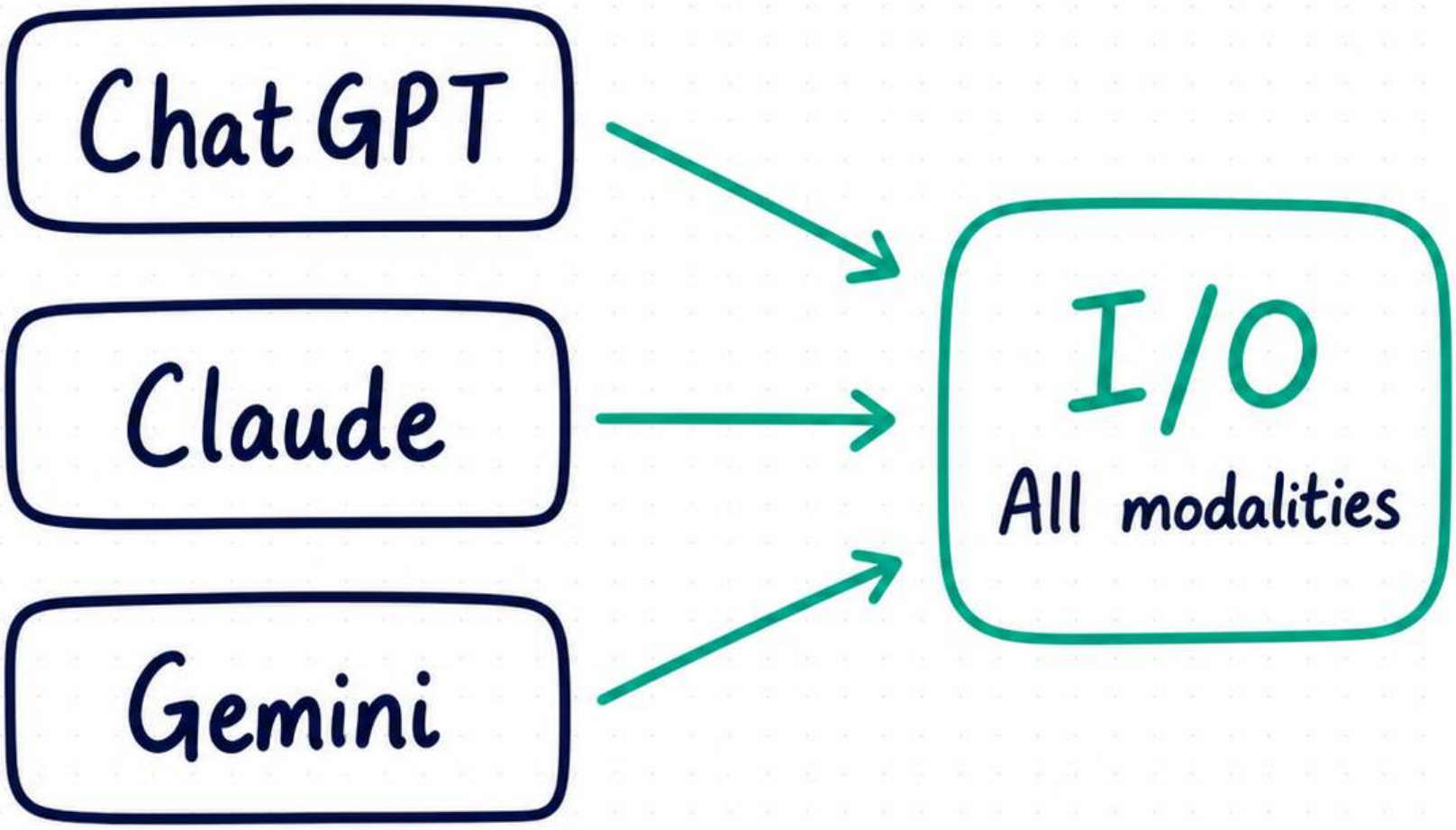
We can do a lot more

if LLMs can understand

all kinds of data



Examples



Speech-to-Text

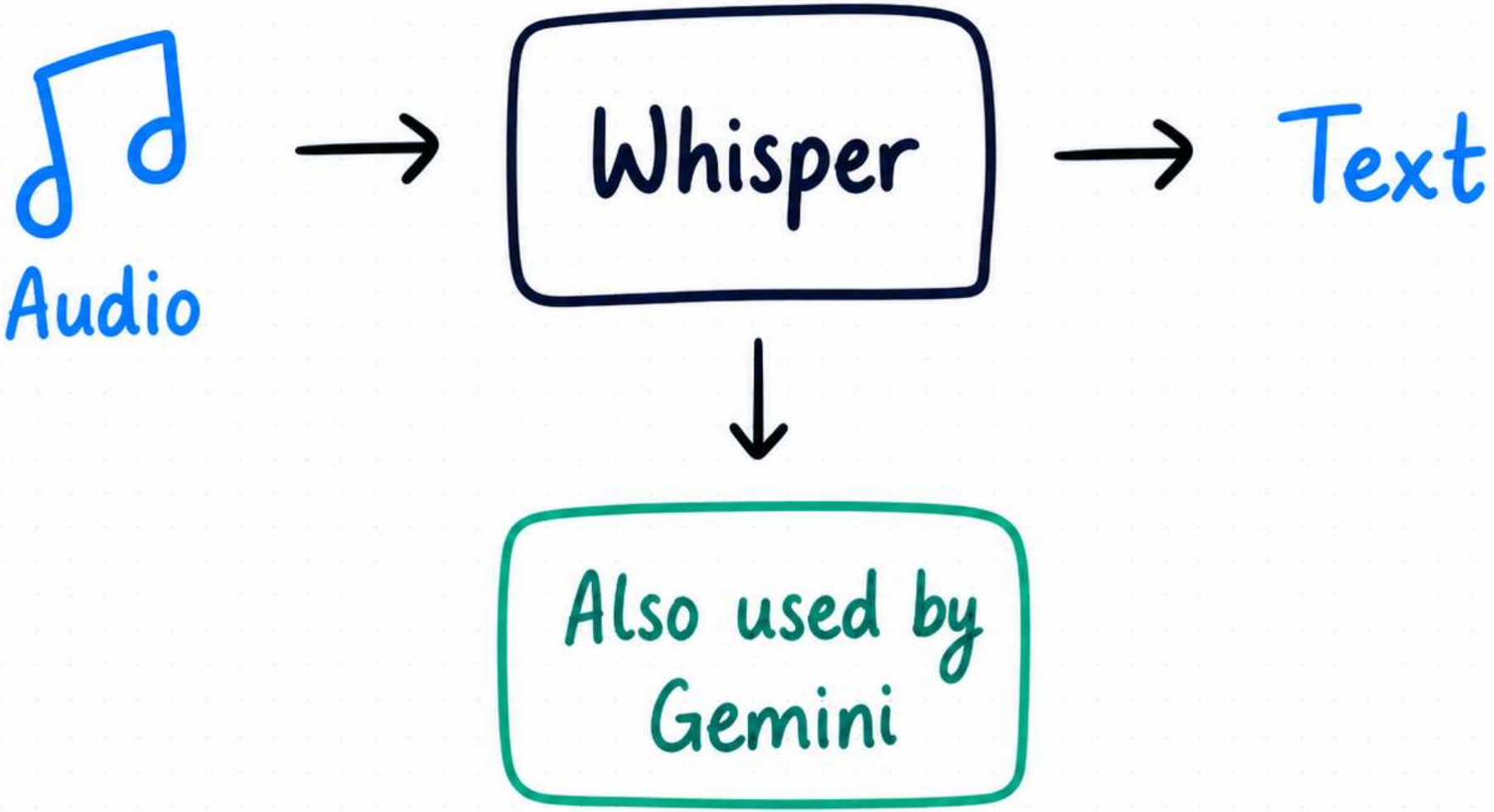
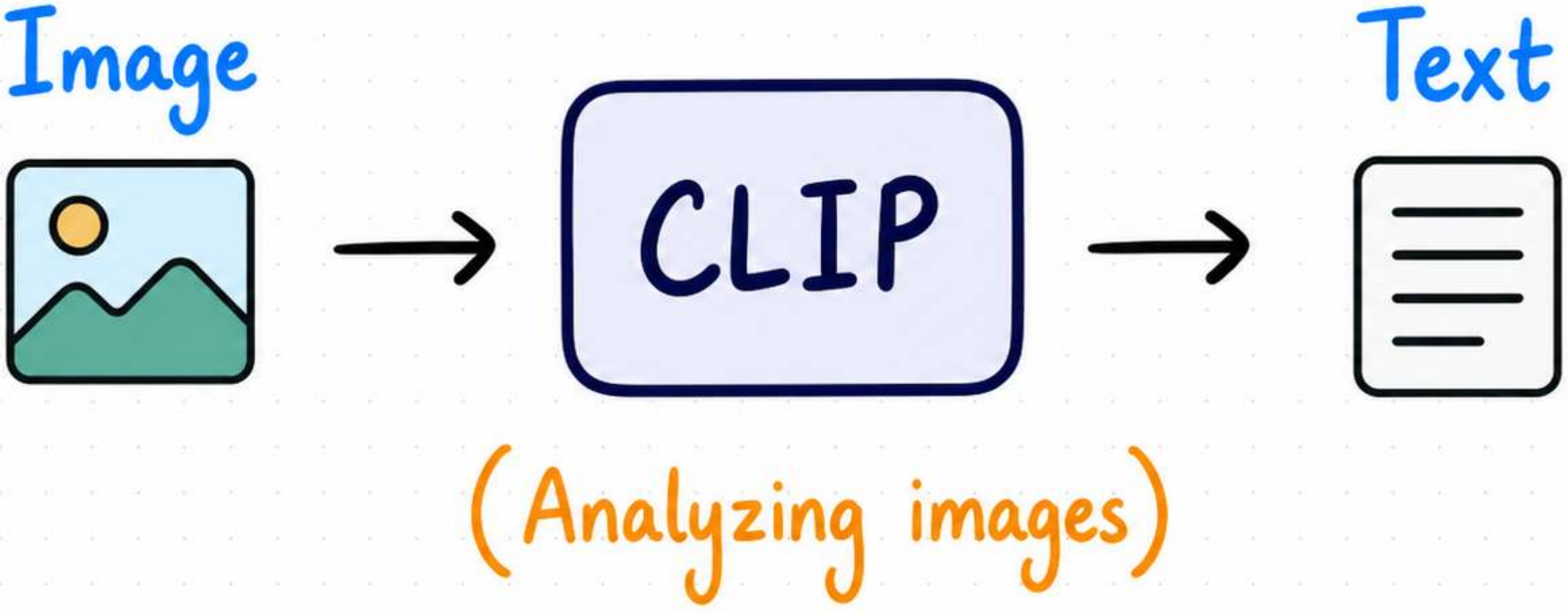
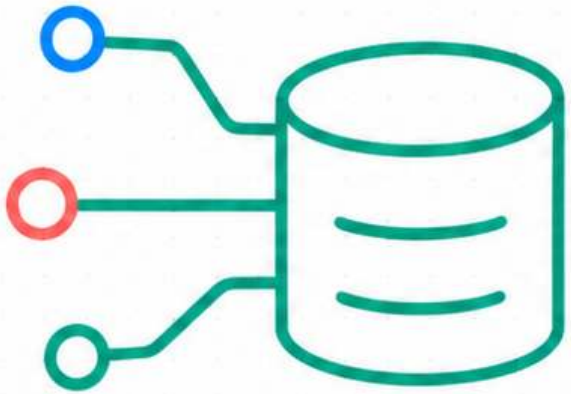


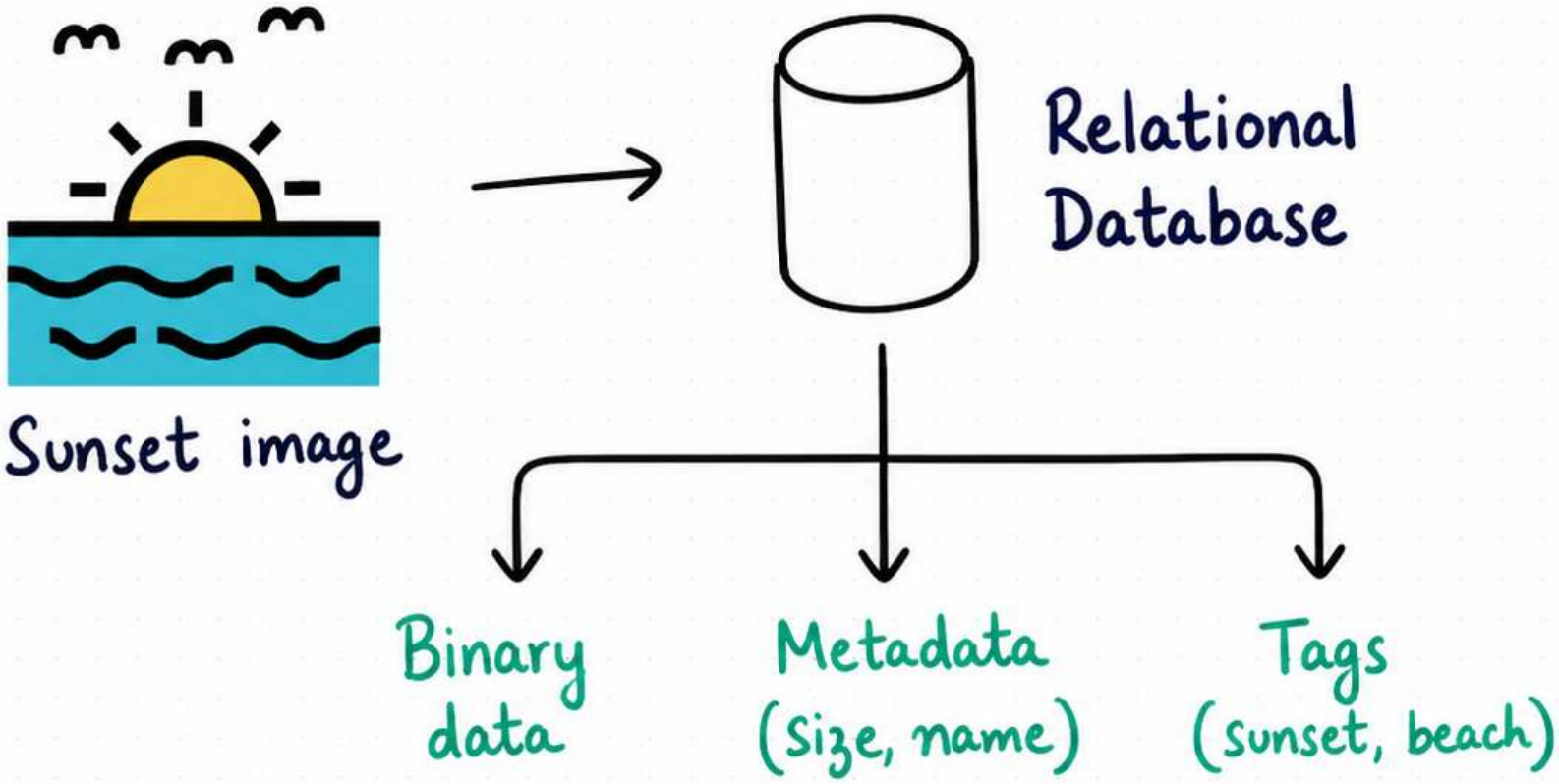
Image-to-Text



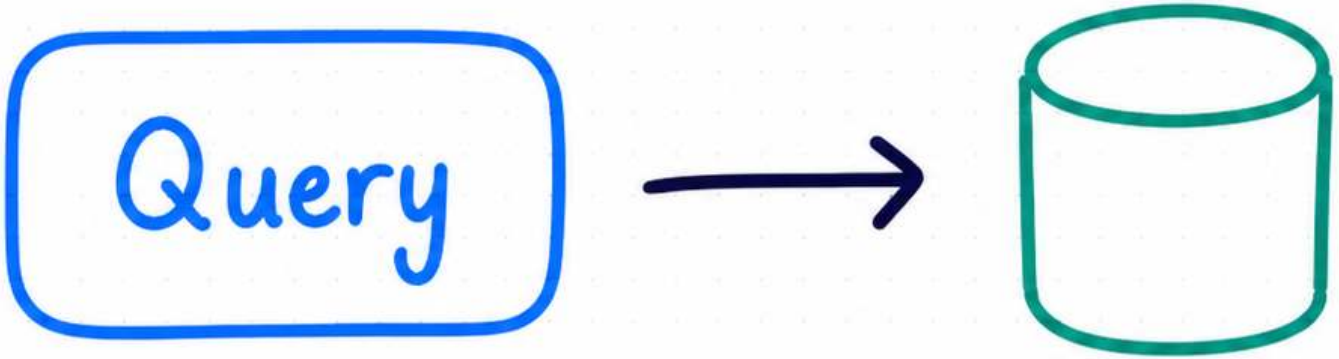


Vector DB

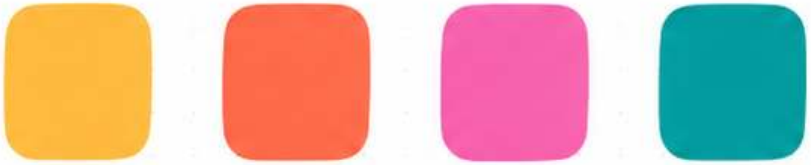
Relational Databases

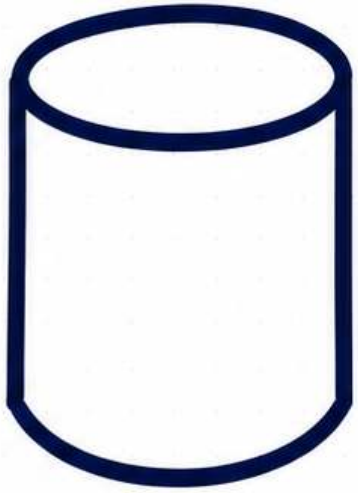


Semantic Query



Get images with the same color palette as the sunset picture





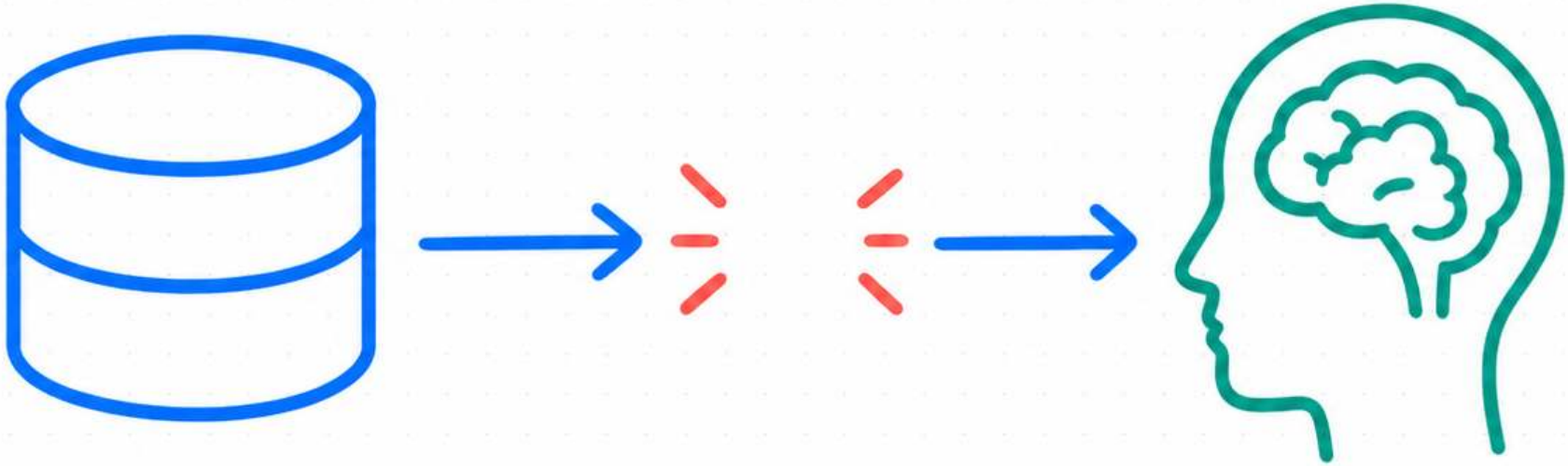
Relational DB





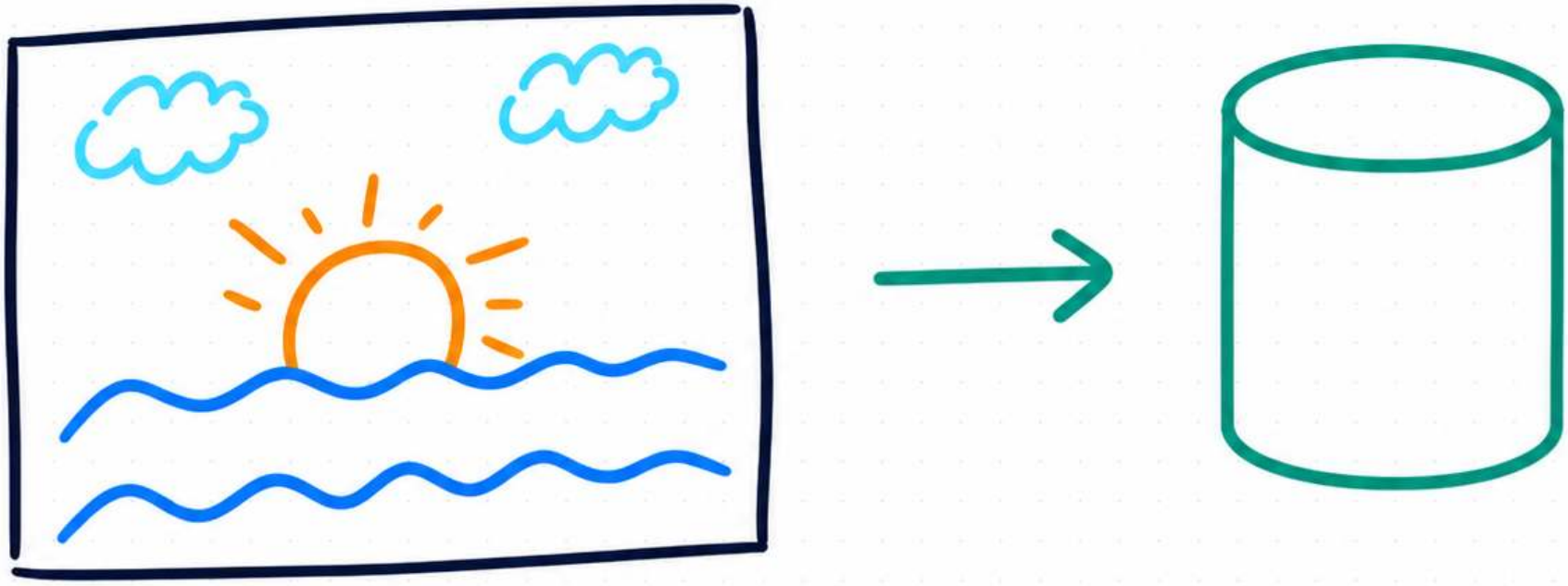
Not good for storing unstructured data

Semantic Gap



Disconnect between how computers store data and how humans understand it

Vector DB



Stores unstructured data such as images, audio, and video

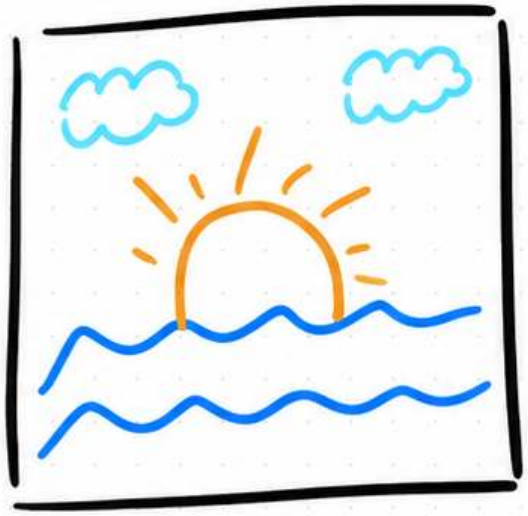
Vector Embeddings

Numerical representation of data

(Image / Audio / Video)

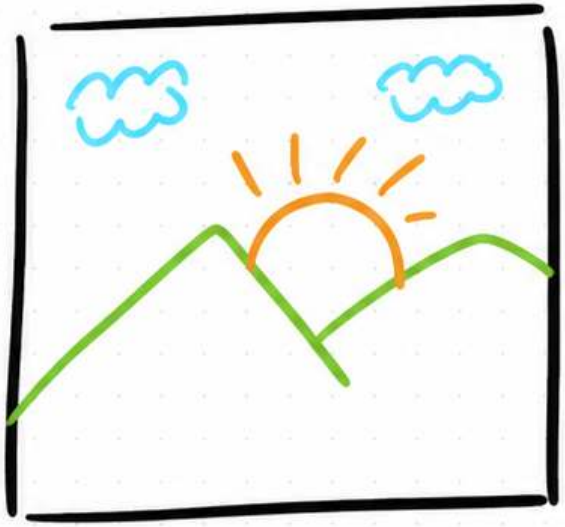
[0.21 , 0.15, 0.83, . . .]

elevation urban warm colors



→ [0.21 , 0.15, 0.83, ---]

elevation urban warm colors



→ [0.95, 0.20, 0.42, ---]

Embedding Models



Data

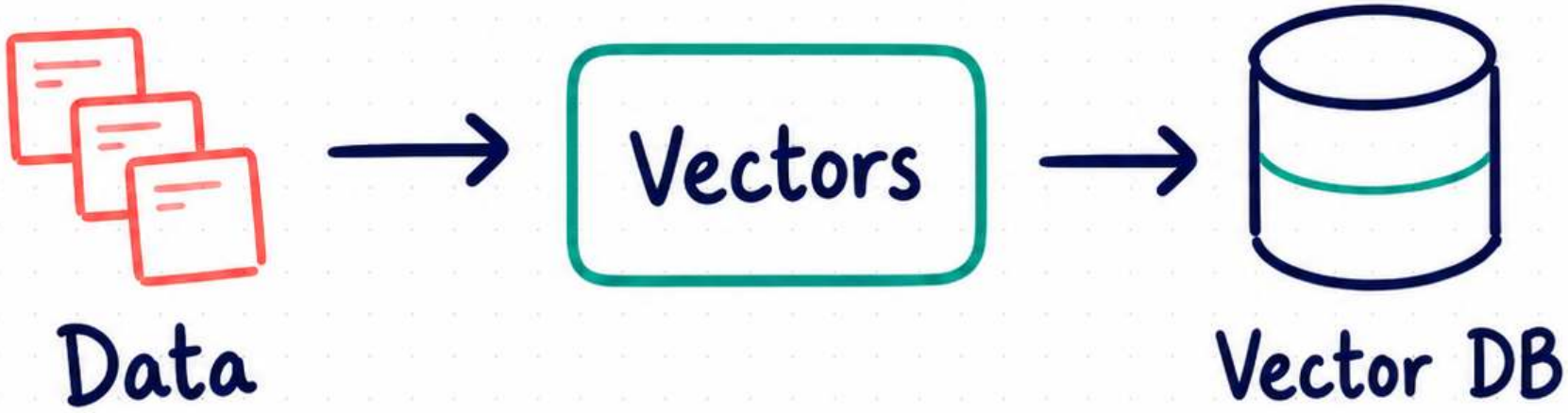
→

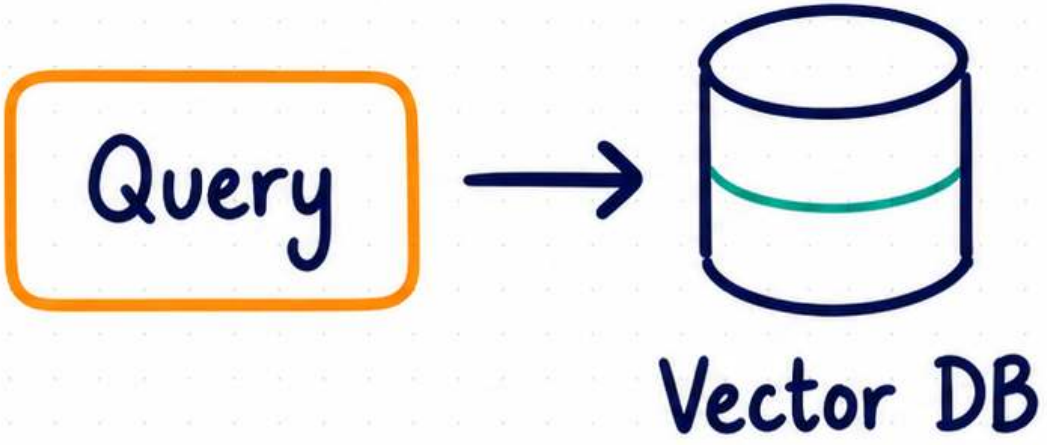
Vectors

Lists of numbers

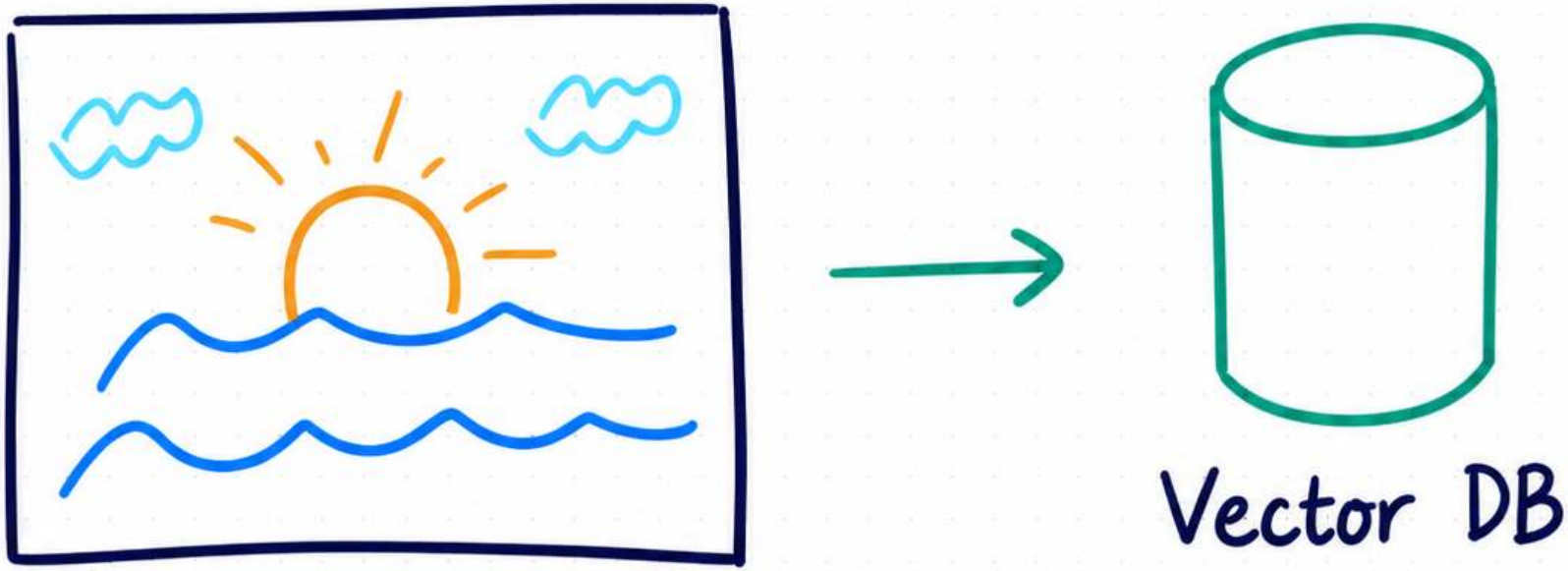
Text	→	Word / sentence embeddings
Image	→	Image embeddings
Audio	→	Speech embeddings

Vector Database



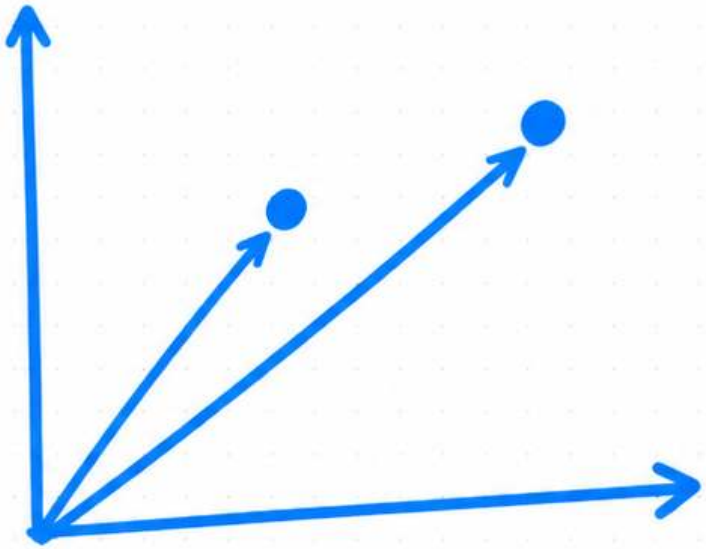


Find an Exact Match



✗ Too slow

Vector Indexing



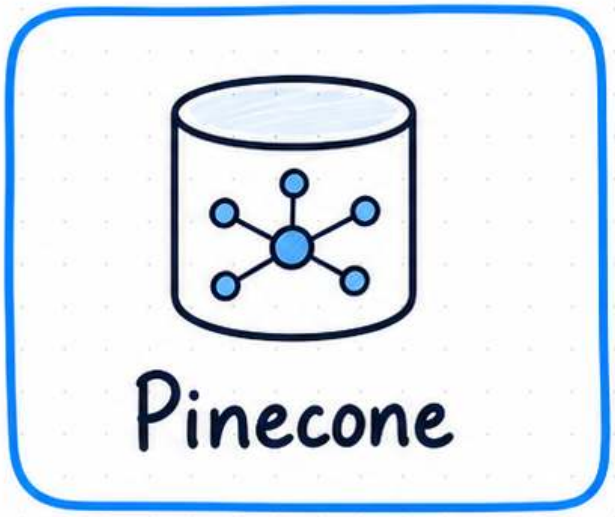
ANN algorithm is used to find the closest match

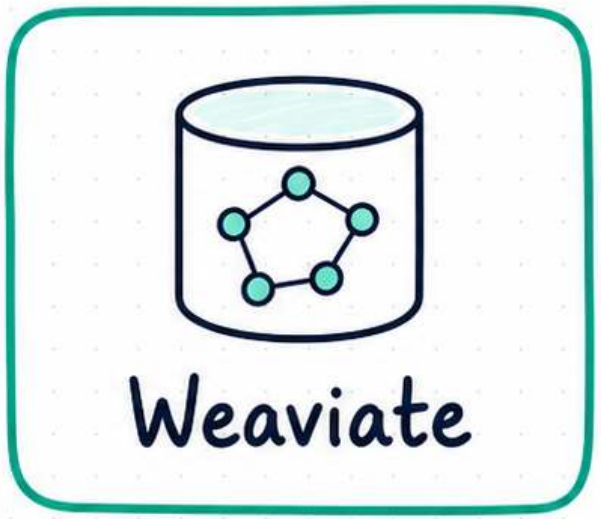
Use Cases

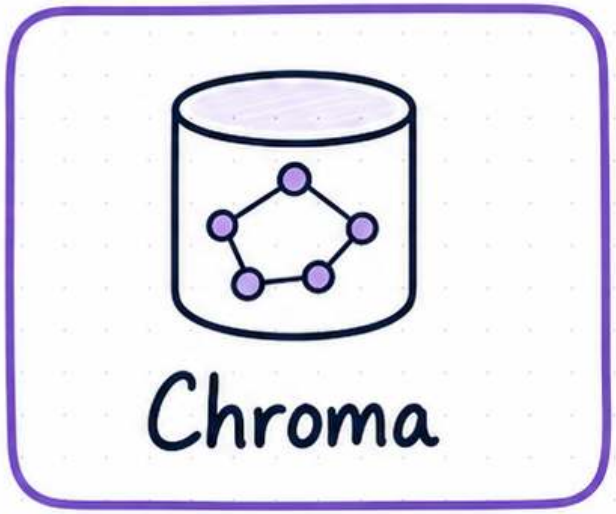
- ✓  Finding a painting of a sunset on a beach
- ✓  Finding a similar dress
- ✓ 

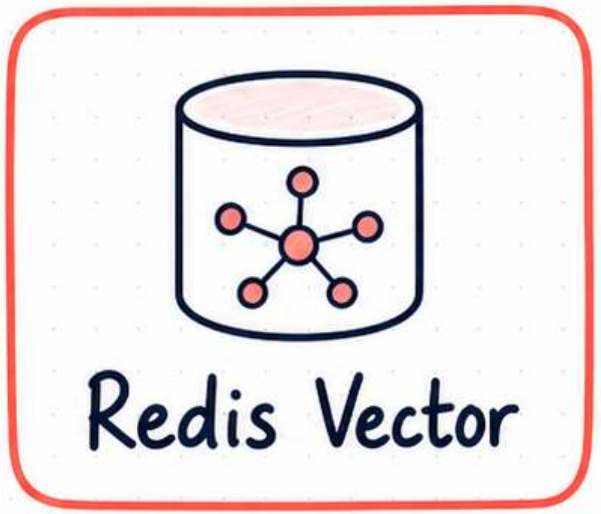
Recommendations for similar hotels

Popular Vector DBs







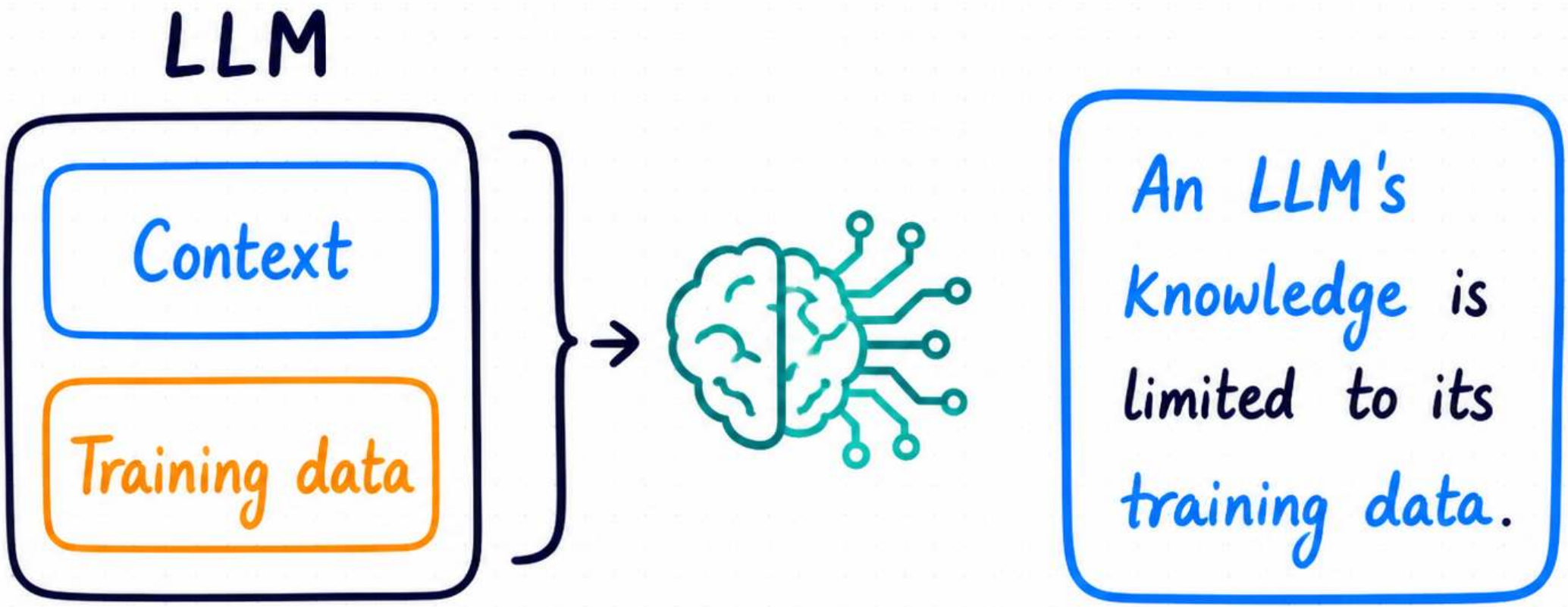




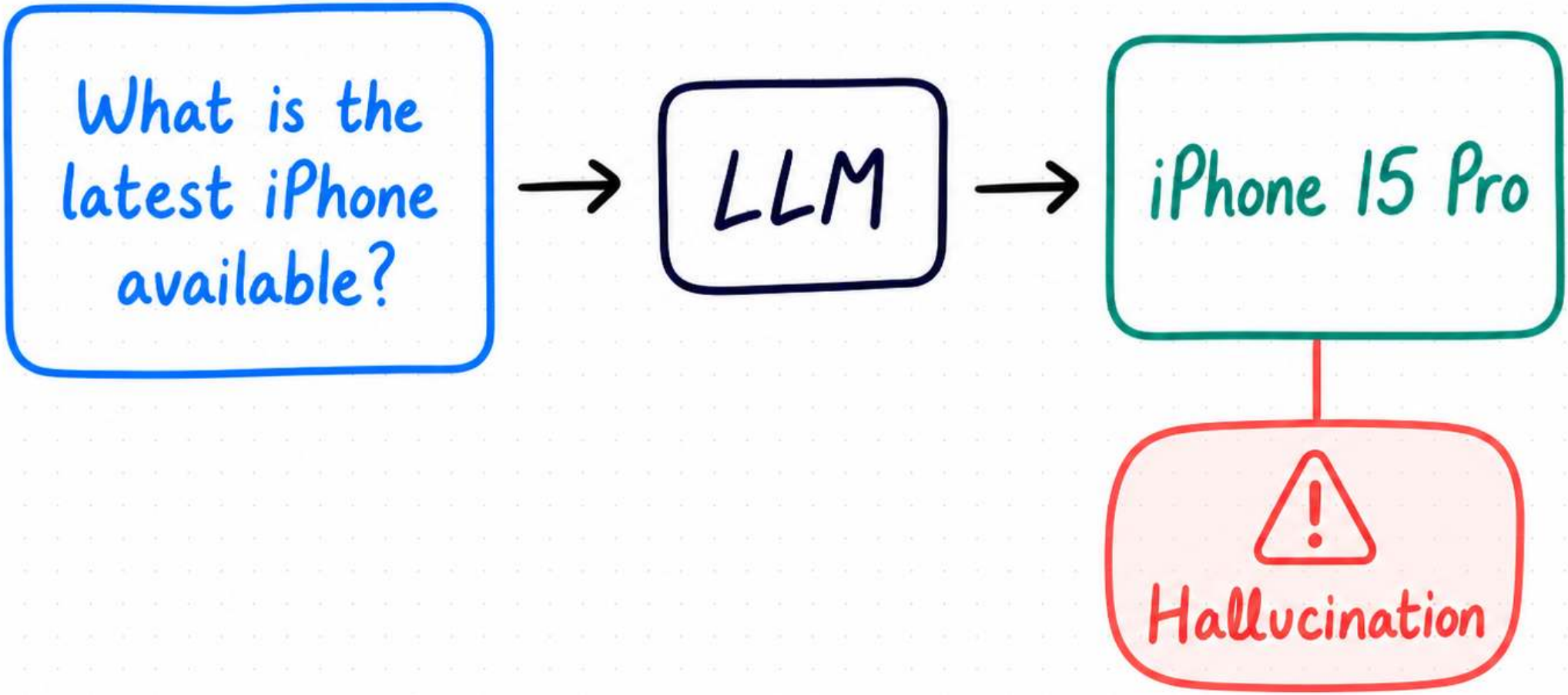
RAG

Retrieval-Augmented Generation

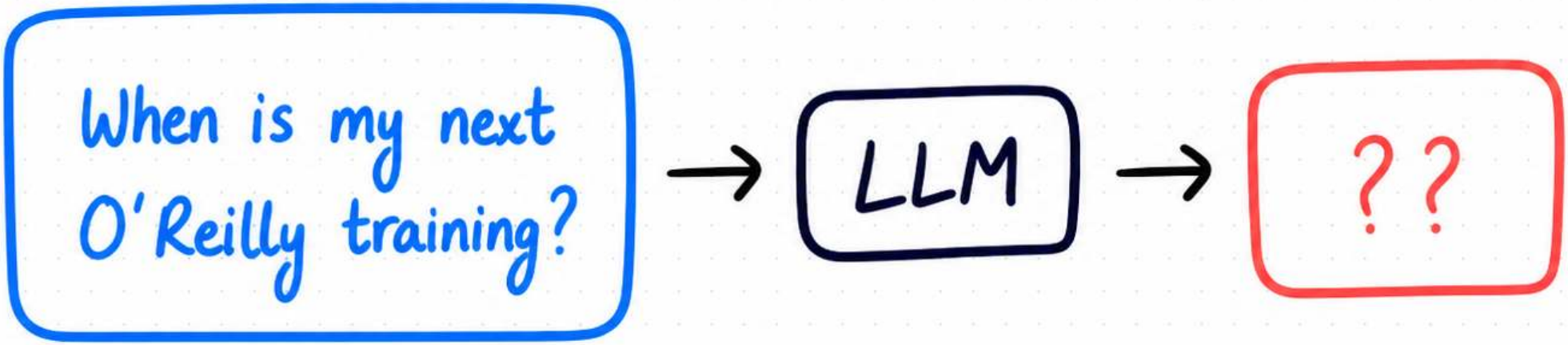
LLM Knowledge Limits



Stale Knowledge



Private Knowledge Gap

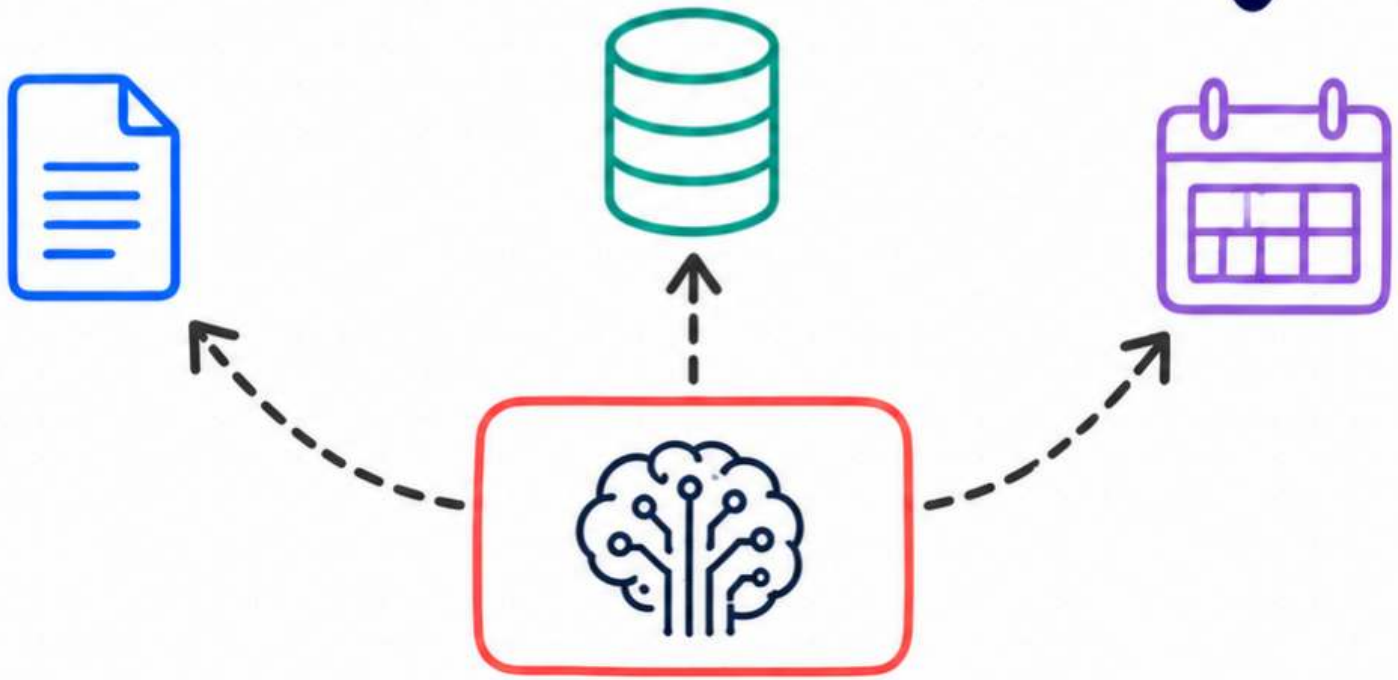


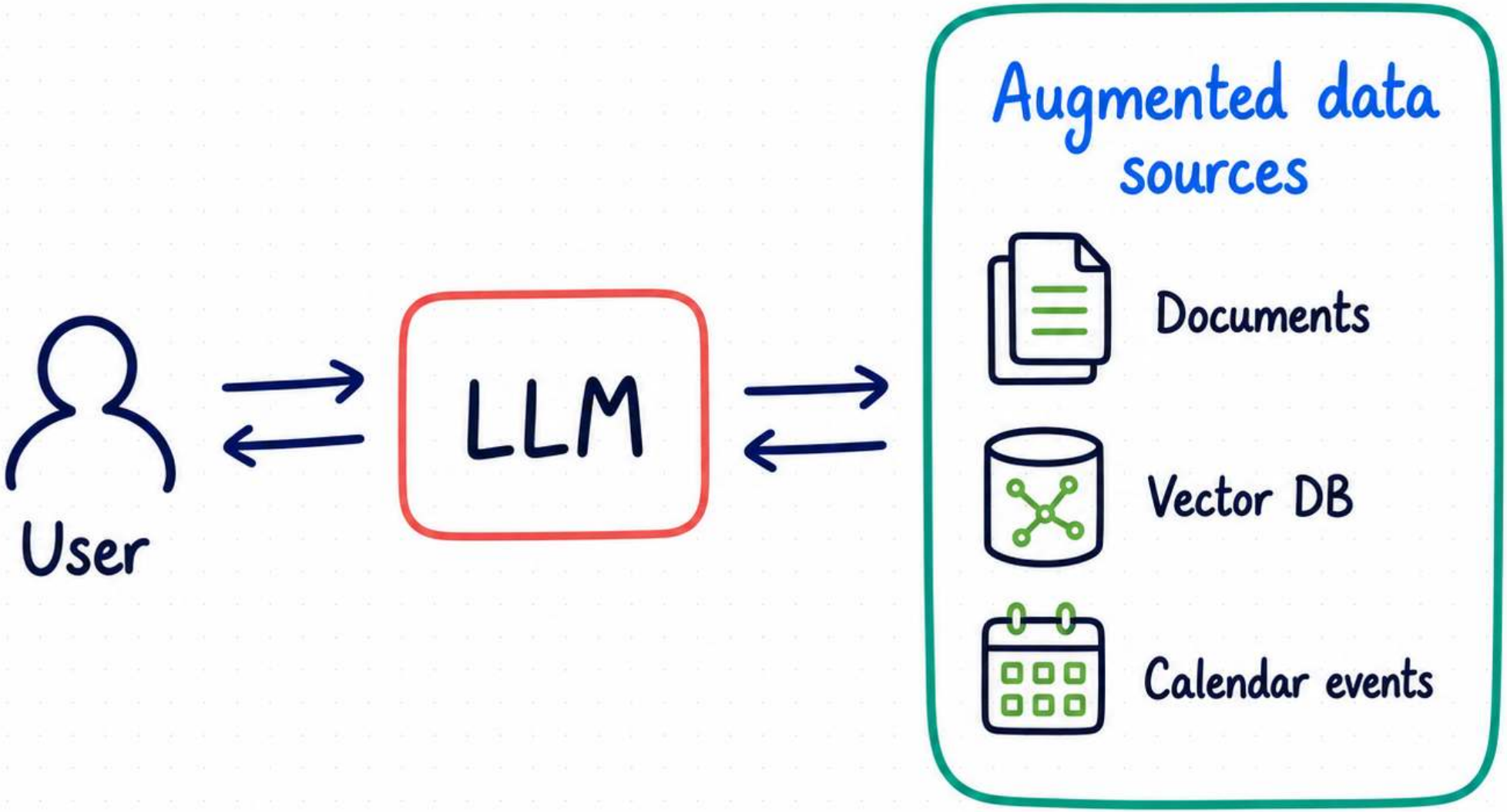
Challenges

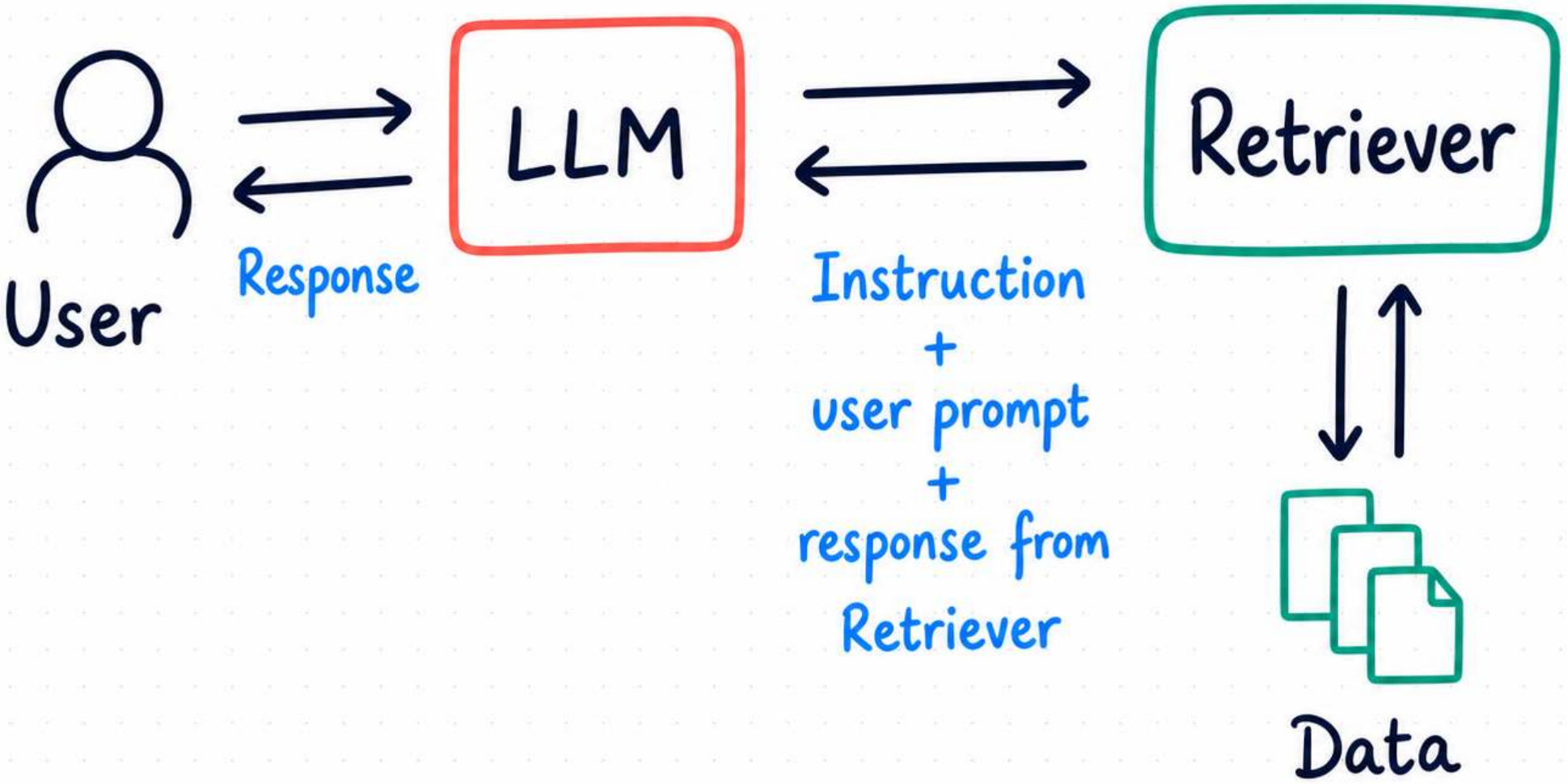
1.  Out of date

2.  No source

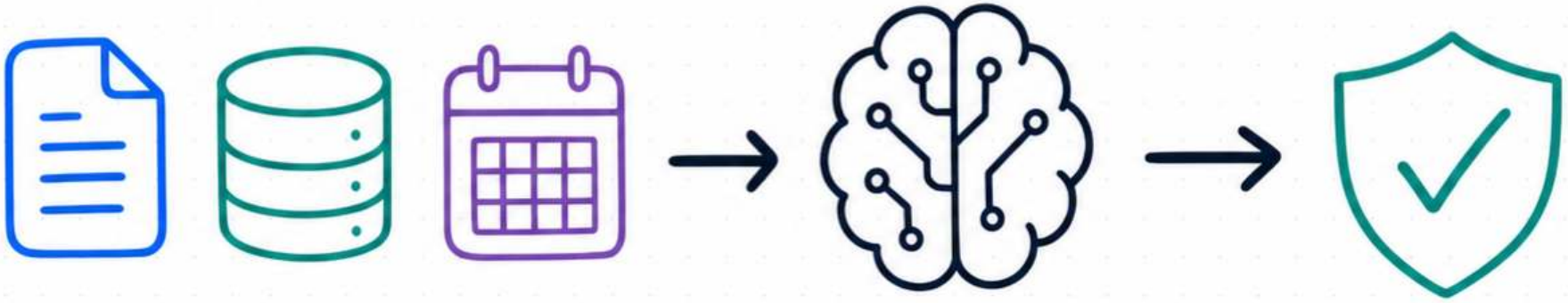
How can we extend an LLM's knowledge?




User
LLM
Augmented data sources
Documents
Vector DB
Calendar events


User
Response
LLM
Instruction
+
user prompt
+
response from
Retriever
Retriever
Data

By extending the source,
LLMs are less likely
to hallucinate.



Pros

- ✓  Latest information from external sources
- ✓  Extended knowledge without increasing the size of the LLM

Limitations

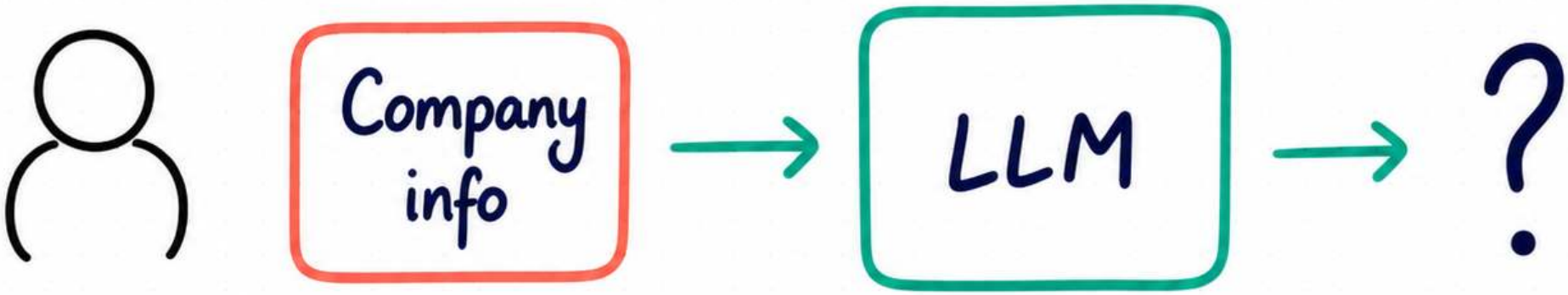
- Two-step pipeline



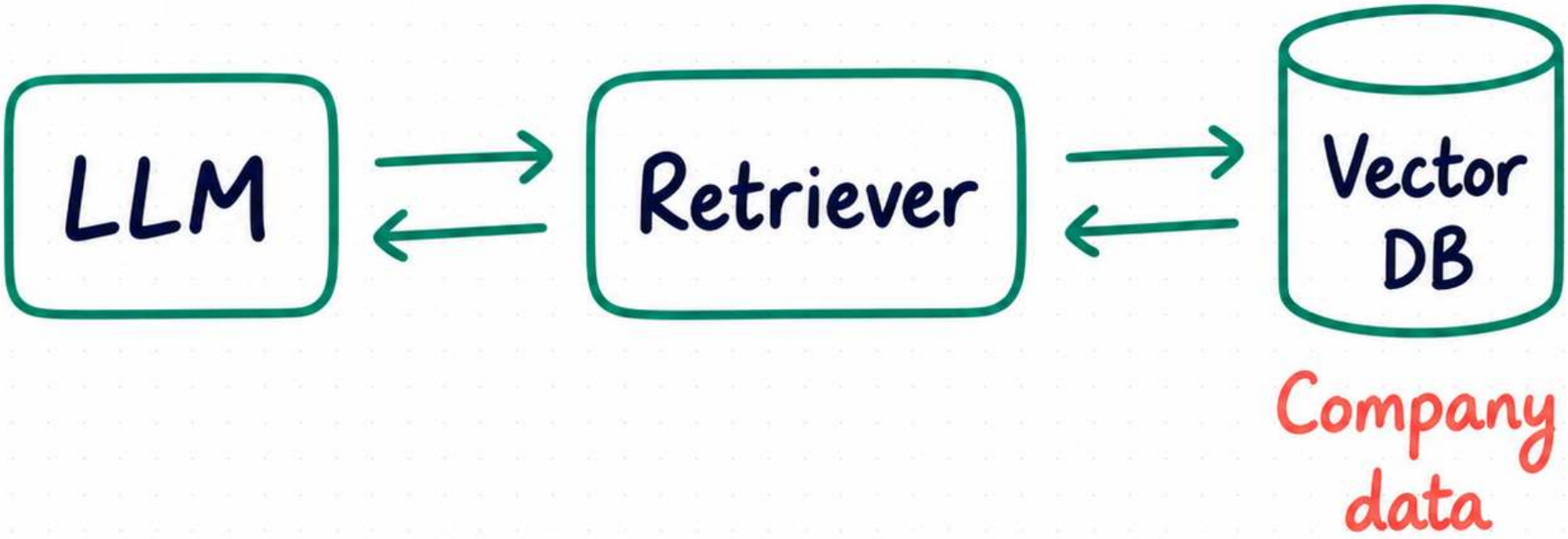
⚠ Errors in retrieval lead to bad answers

CAG

Context-Augmented Generation

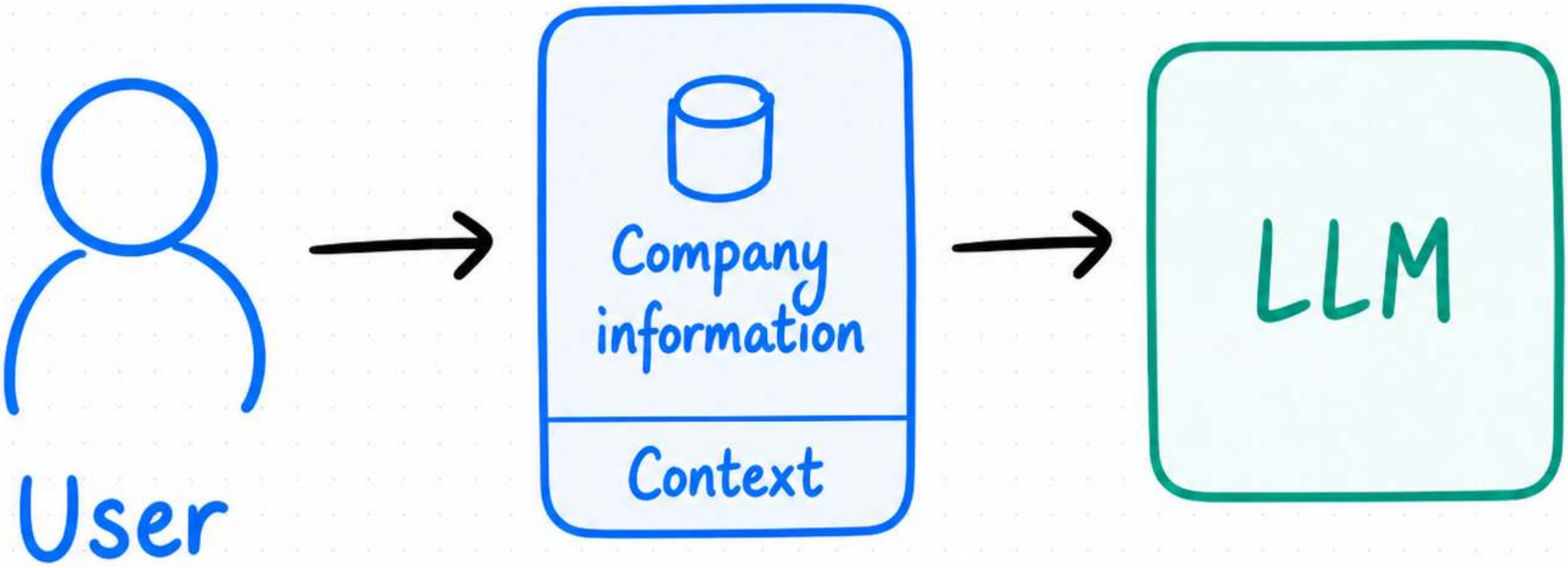

Company info
LLM
?

Using RAG



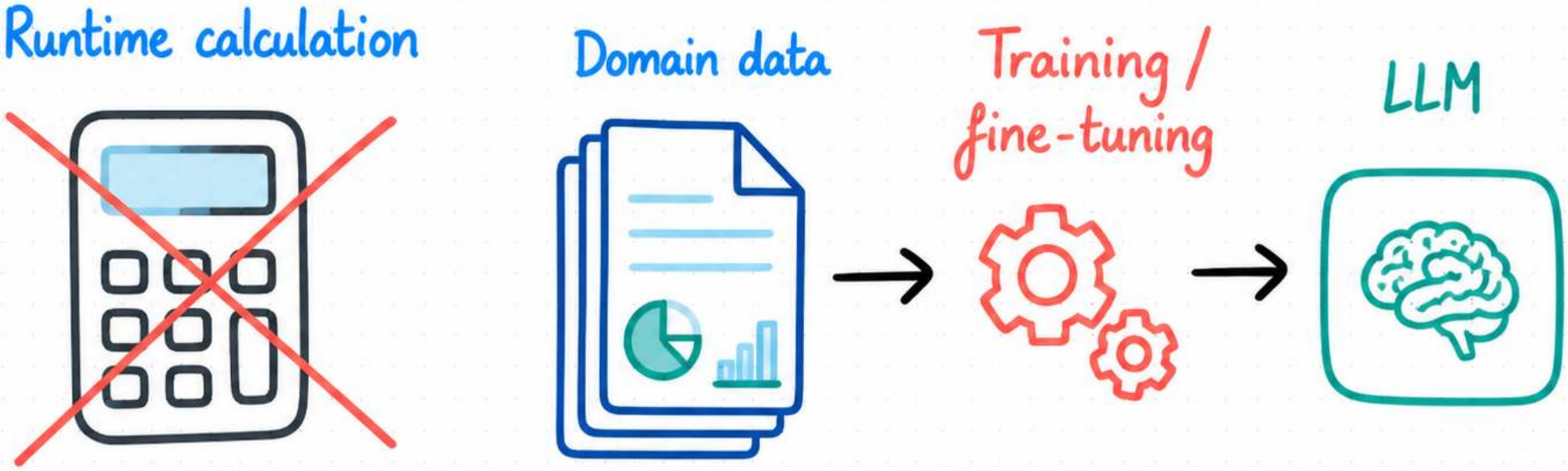
Direct Context

Provide company information directly to the LLM



Domain Fine-Tuning

Instead of calculating data at runtime, the LLM is trained or fine-tuned on a specific domain.



Pros

- ✓  Faster
- ✓  Easy to implement for small knowledge sets

Limitations



Context size is limited

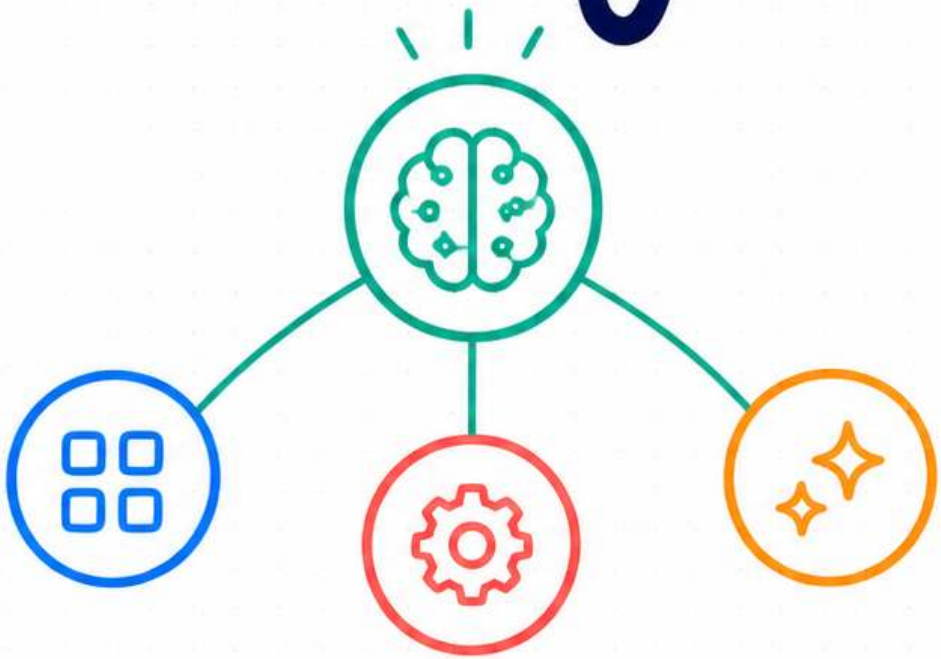


Needs frequent updating if data changes



Difficult to scale with large datasets

AI Agents

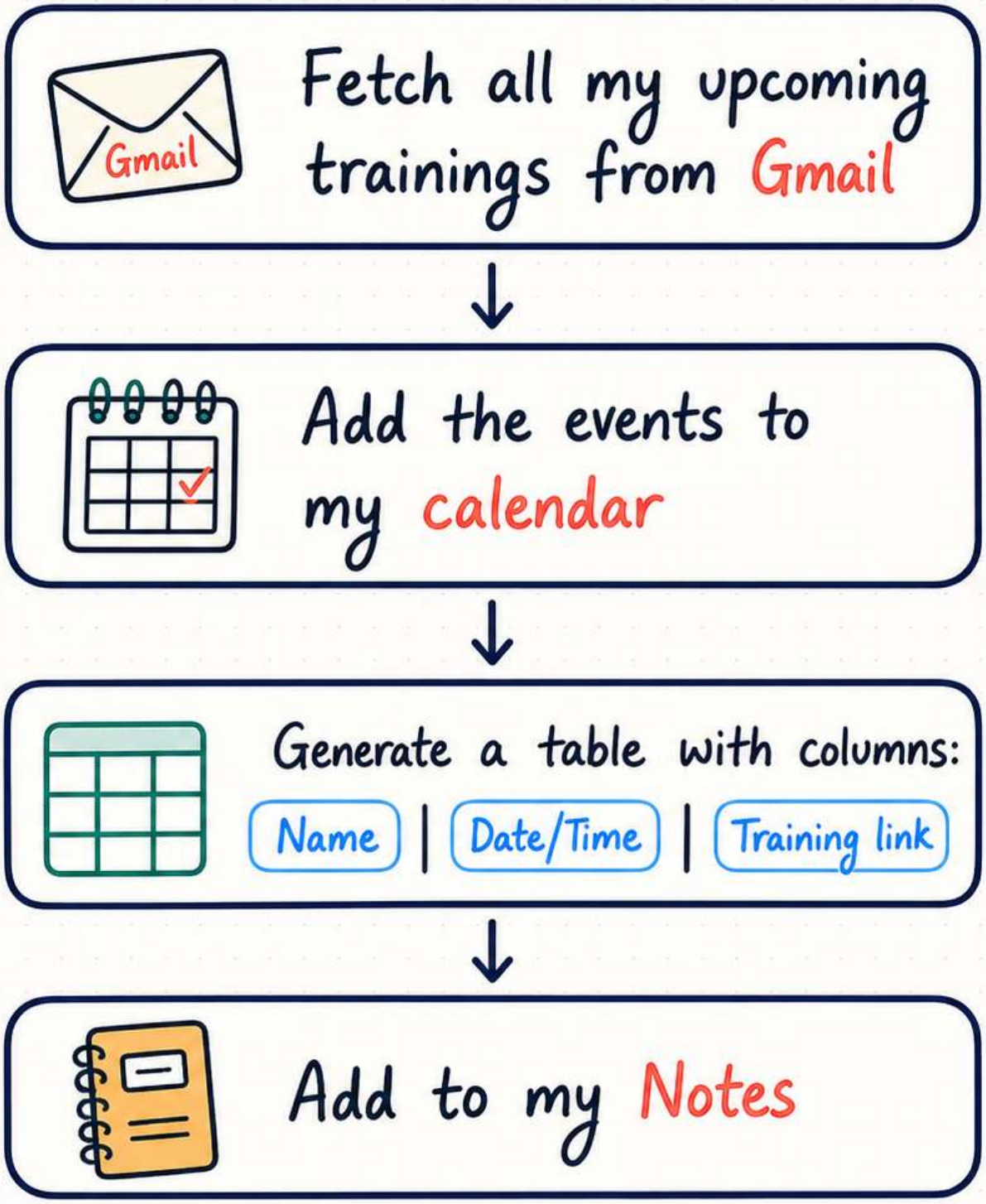


LLMs are limited

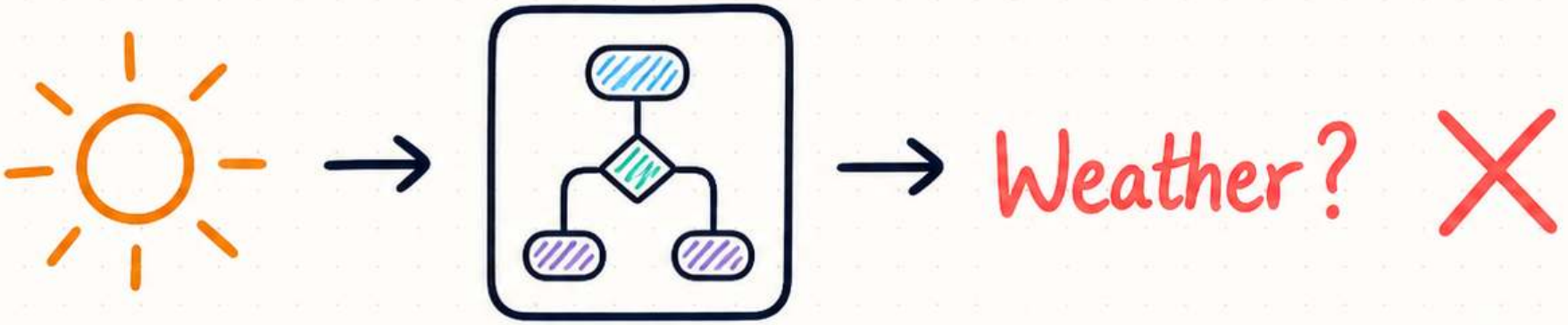
1. Limited to their knowledge 
2. They are passive (wait for user action) 
3. Can't act

Create event → 


AI Workflows


Gmail
Fetch all my upcoming trainings from Gmail
Add the events to my calendar
Generate a table with columns:
Name | Date/Time | Training link
Add to my Notes

Workflows extend the capabilities of LLMs, but they follow predefined paths set by humans.

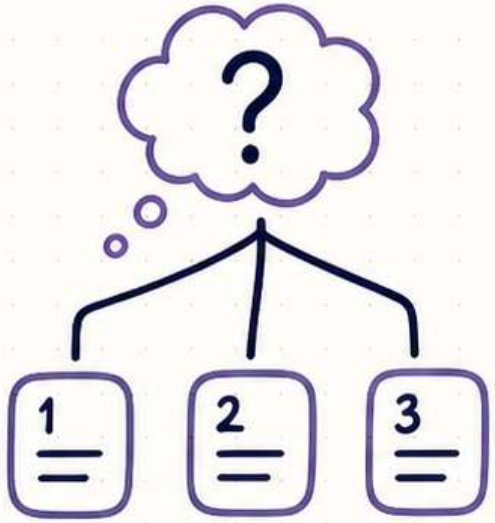


AI Workflow ✗

Agents

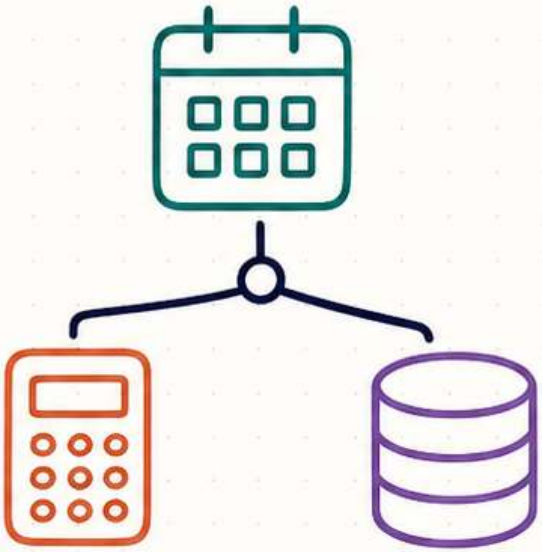
1 Reasoning

- Break down the problem
- Make a plan



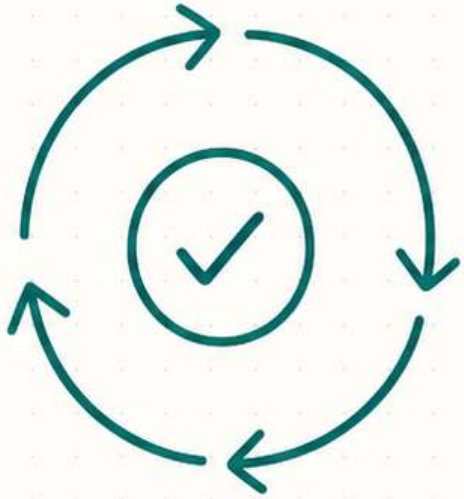
2 Action

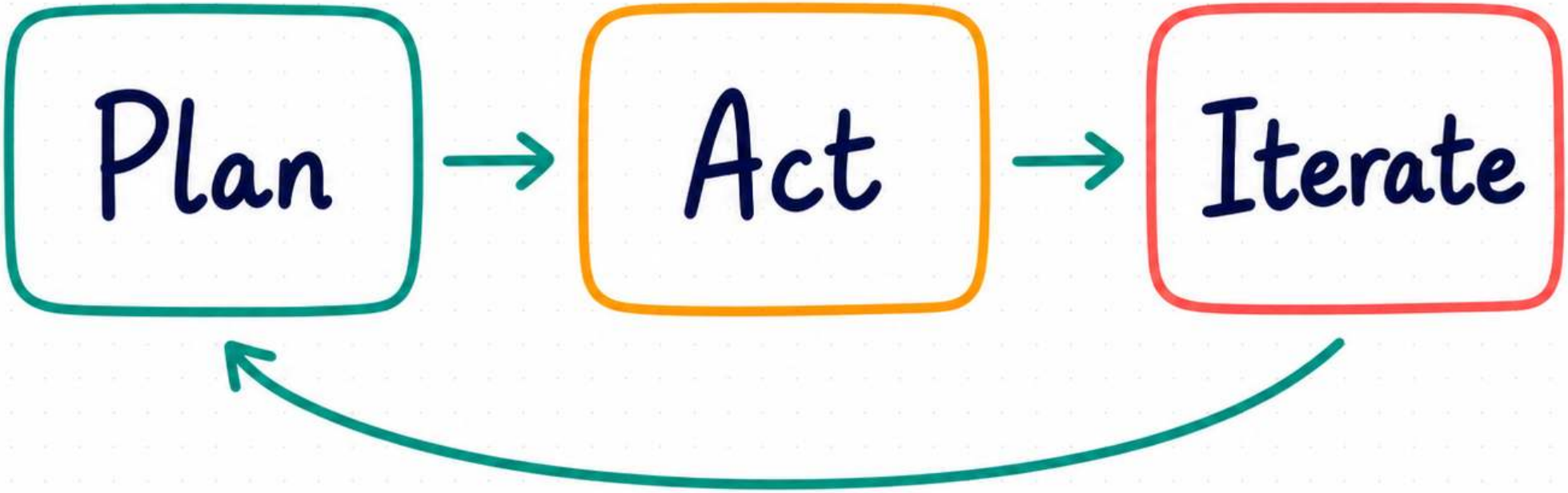
Use external tools like calendars, calculators, and databases



3 Iterate

Repeat the steps if the problem is not solved in one way




Plan
Act
Iterate

Schedule training for Dec 5



Agent





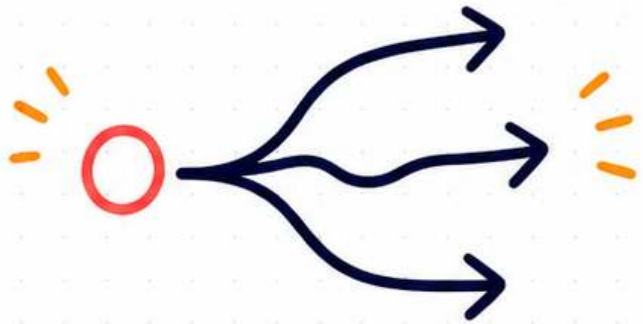
1. Check emails and calendar for existing courses
2. Check for a course on Dec 5

No course found

3. Add event to my calendar
4. Send confirmation email to producer

Agents

Flexible, adaptive, and well-structured for open-ended tasks



Workflows

Efficient, but follow a fixed path



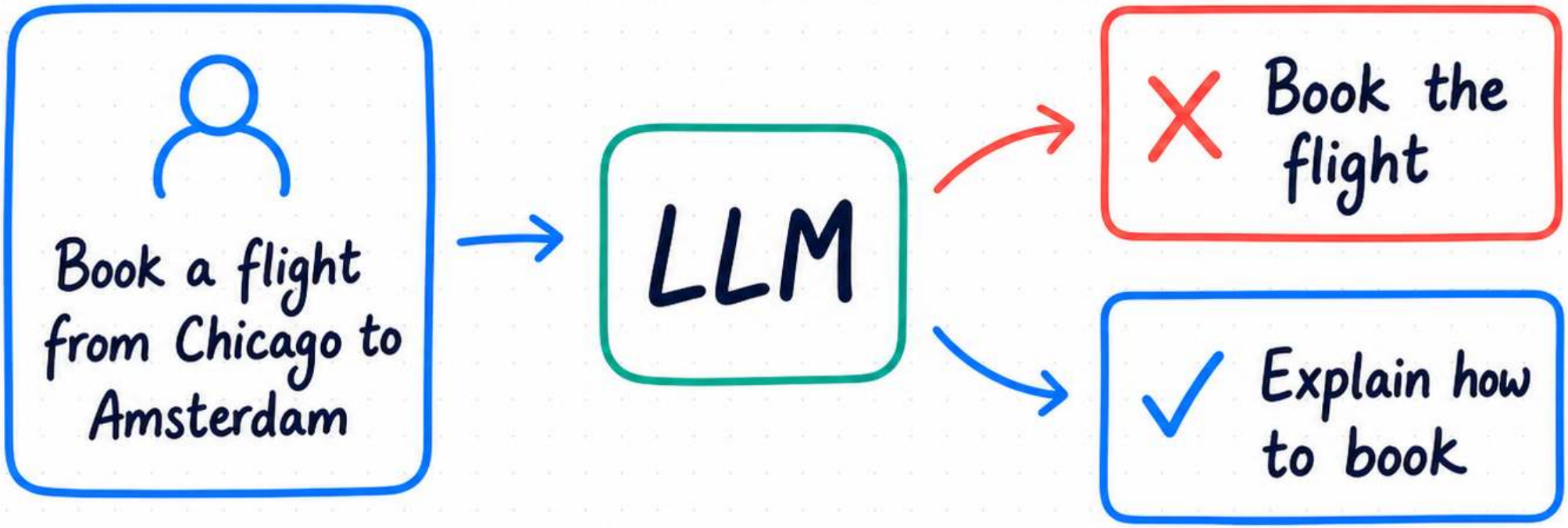
Evolution



MCP

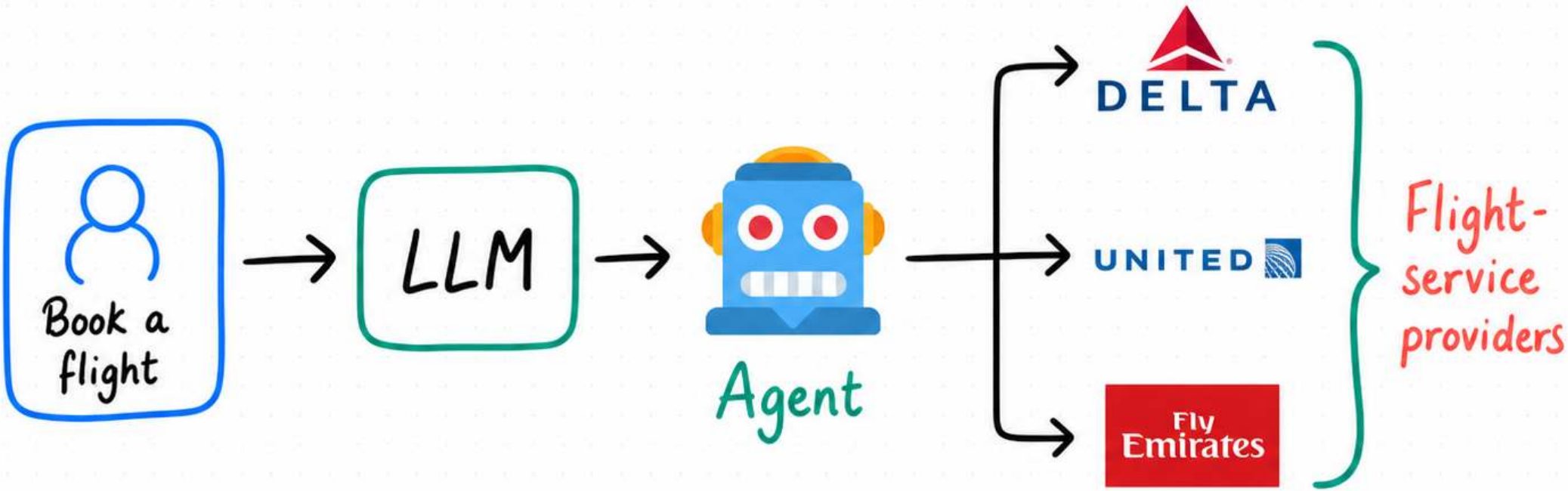
Model Context Protocol

Booking



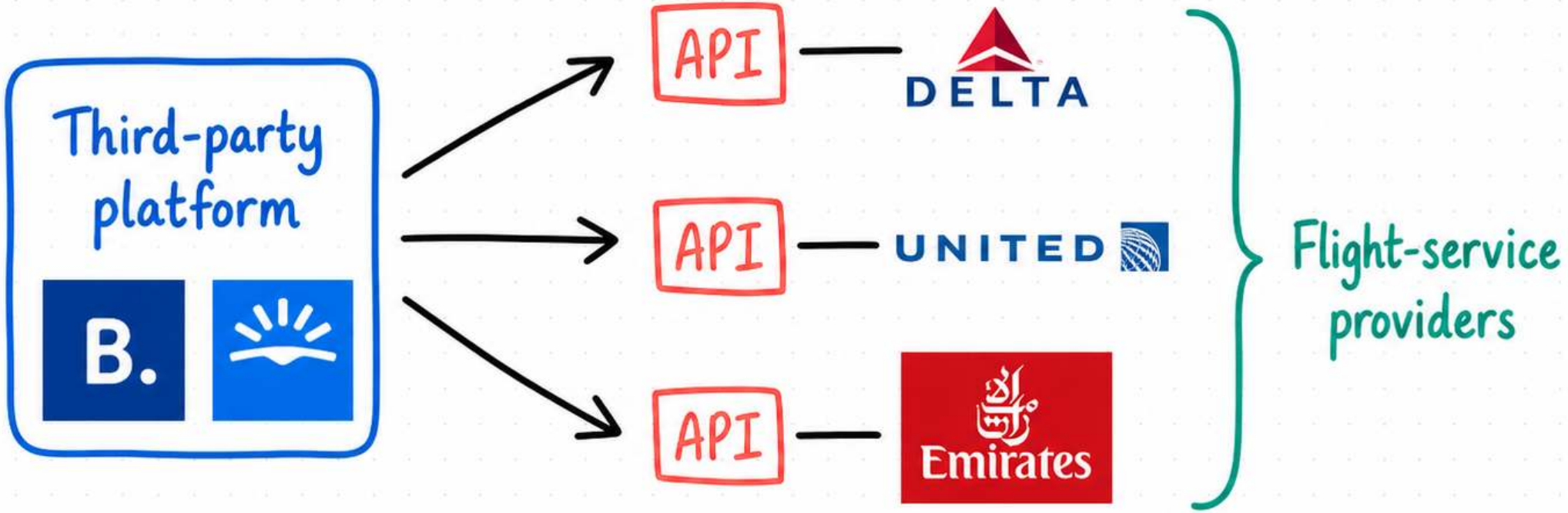
A standard LLM can guide you, but it cannot complete the booking.

LLM + Agent



The agent connects the LLM to booking services.

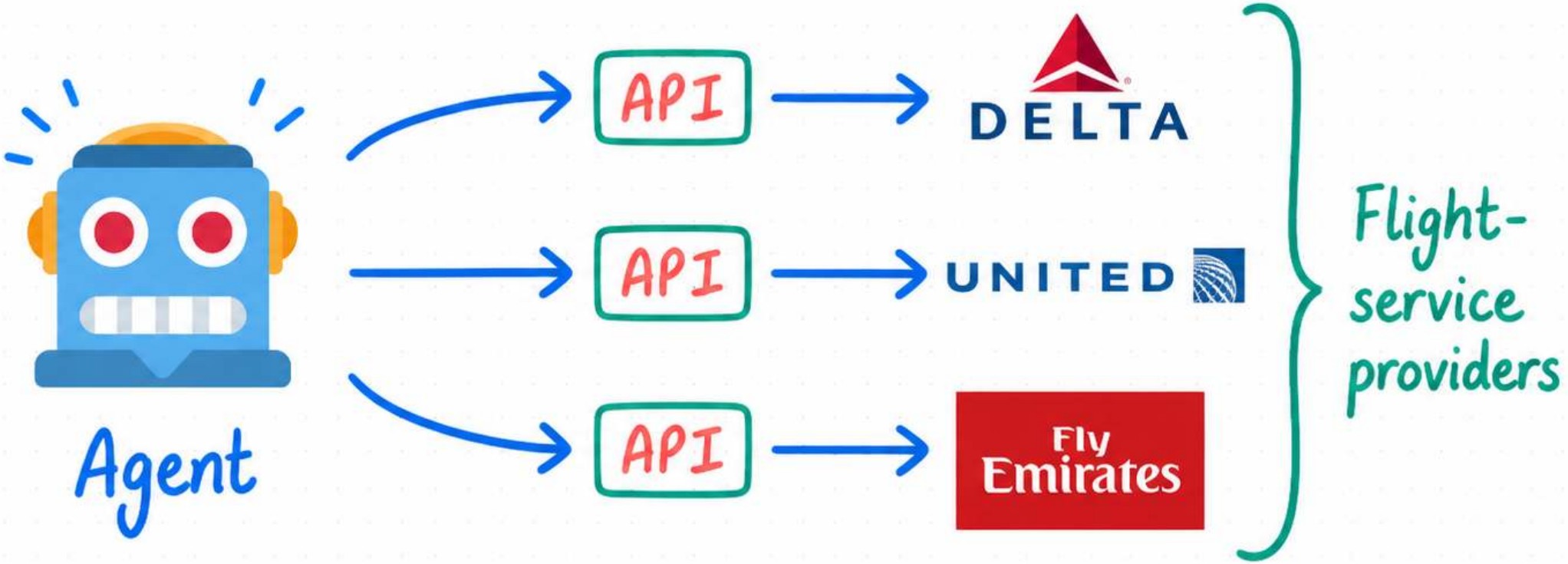
Third-Party Booking Platforms



One platform connects to multiple provider APIs.

Agents Book Flights Directly

Instead of using third-party websites, an agent can book the flight.



Provider-Specific Tools



Agent

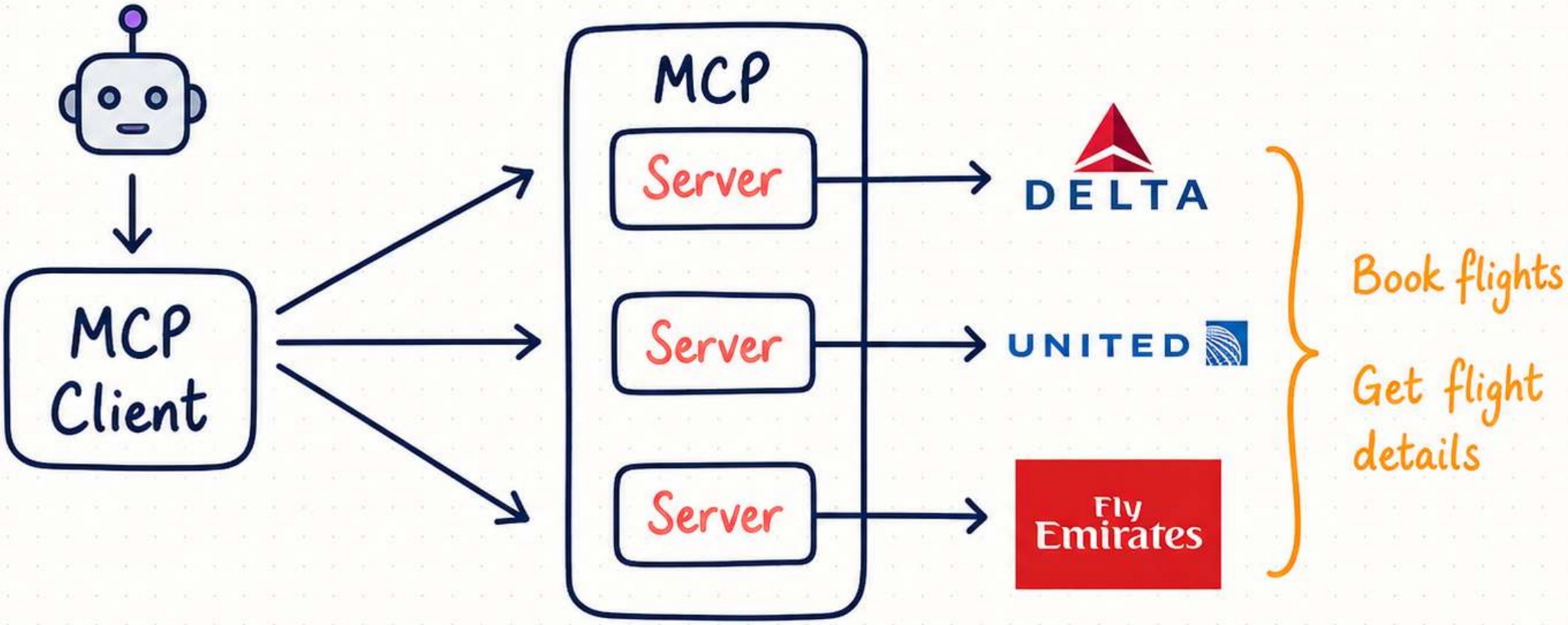
Delta tool → Delta API → DELTA

United tool → United API → UNITED

Emirates tool → Emirates API → Emirates

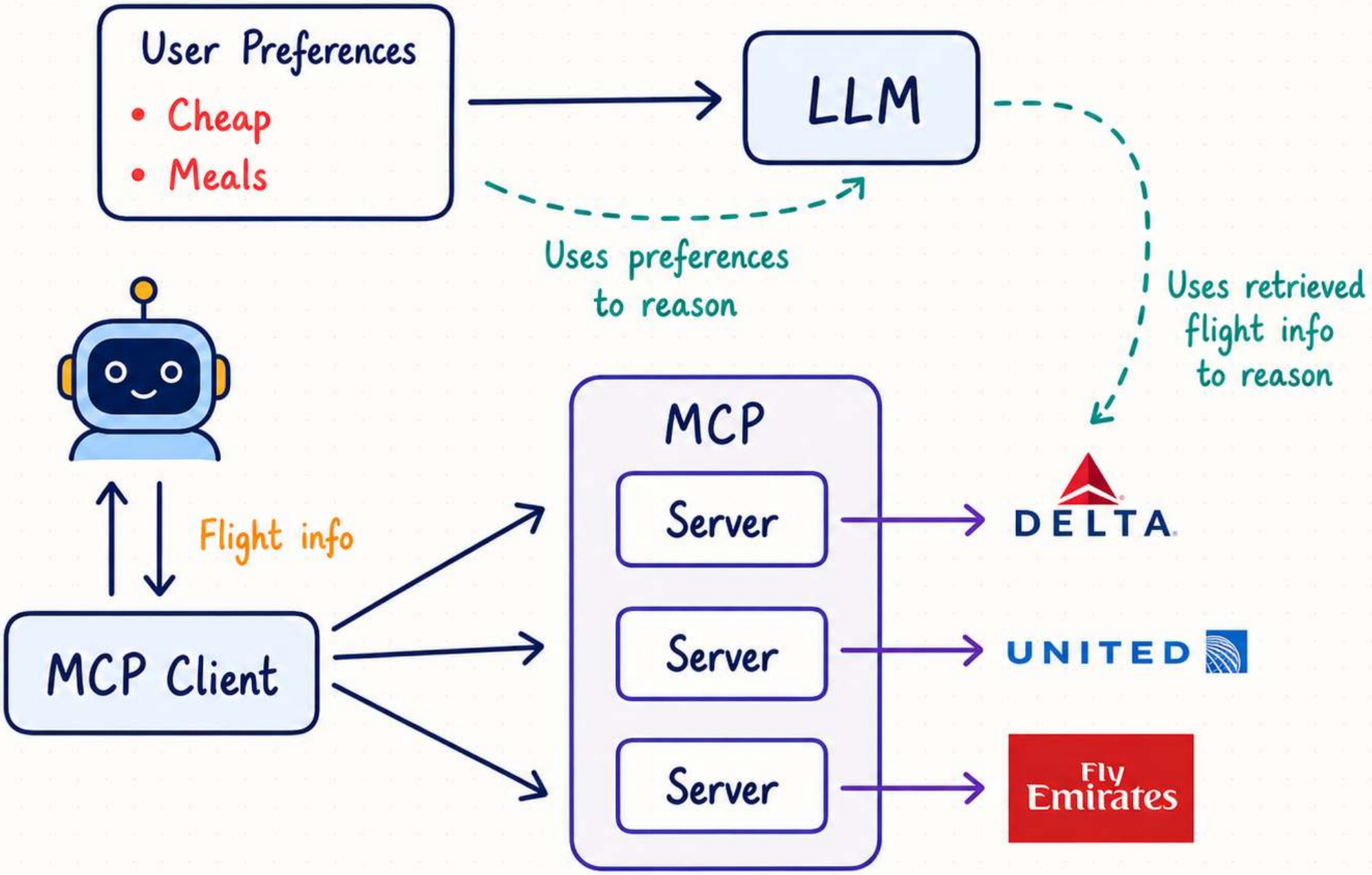
The agent makes three different API calls.

MCP: One Client, Many Servers



Each website exposes its own MCP server. The MCP client talks to each server to get flight details.

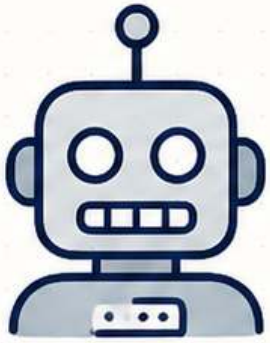
How the Agent Uses MCP



LLM vs Agent

LLM → Decides which flight to book





Agent → Books the flight



Real-World Scenario

⚠ Login functionality is broken in the UI



→ Git History →



Find the broken commit

↓

BE + FE Code

JIRA MCP



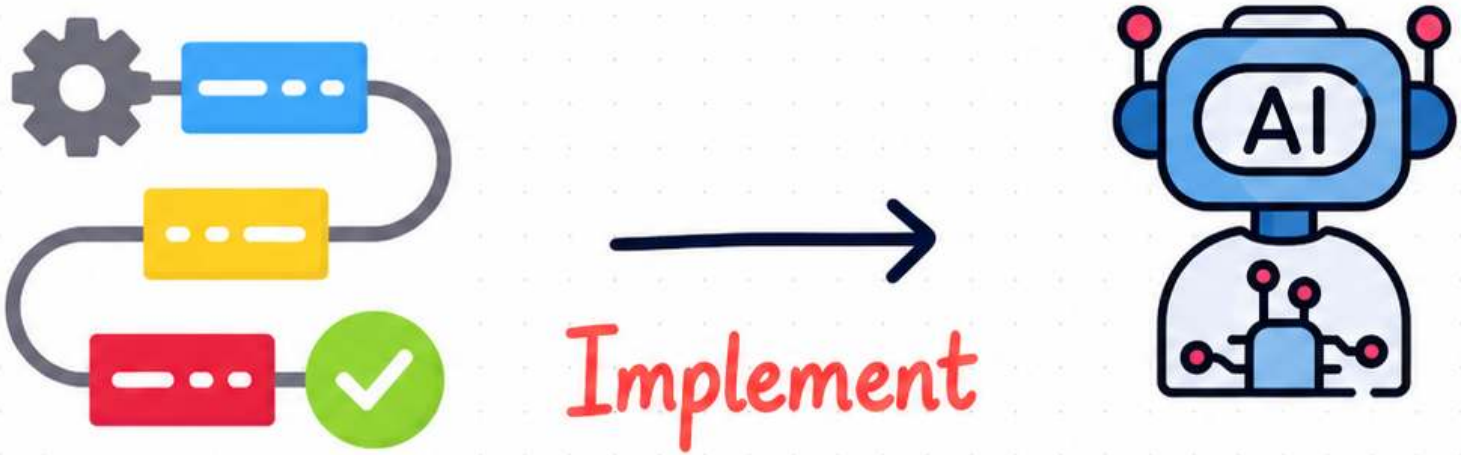
1 Create an issue

2 Get an issue

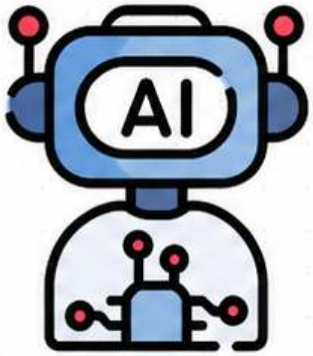
3 Start work on a ticket

SKILLS

Think about a workflow you repeatedly ask your agent to implement.



Create PR



Instructions

- Project ID
- Reviewers
- PR description format

Create branch → Commit → Push → PR description → Assign reviewers

✗ Repeatedly writing long prompts every time

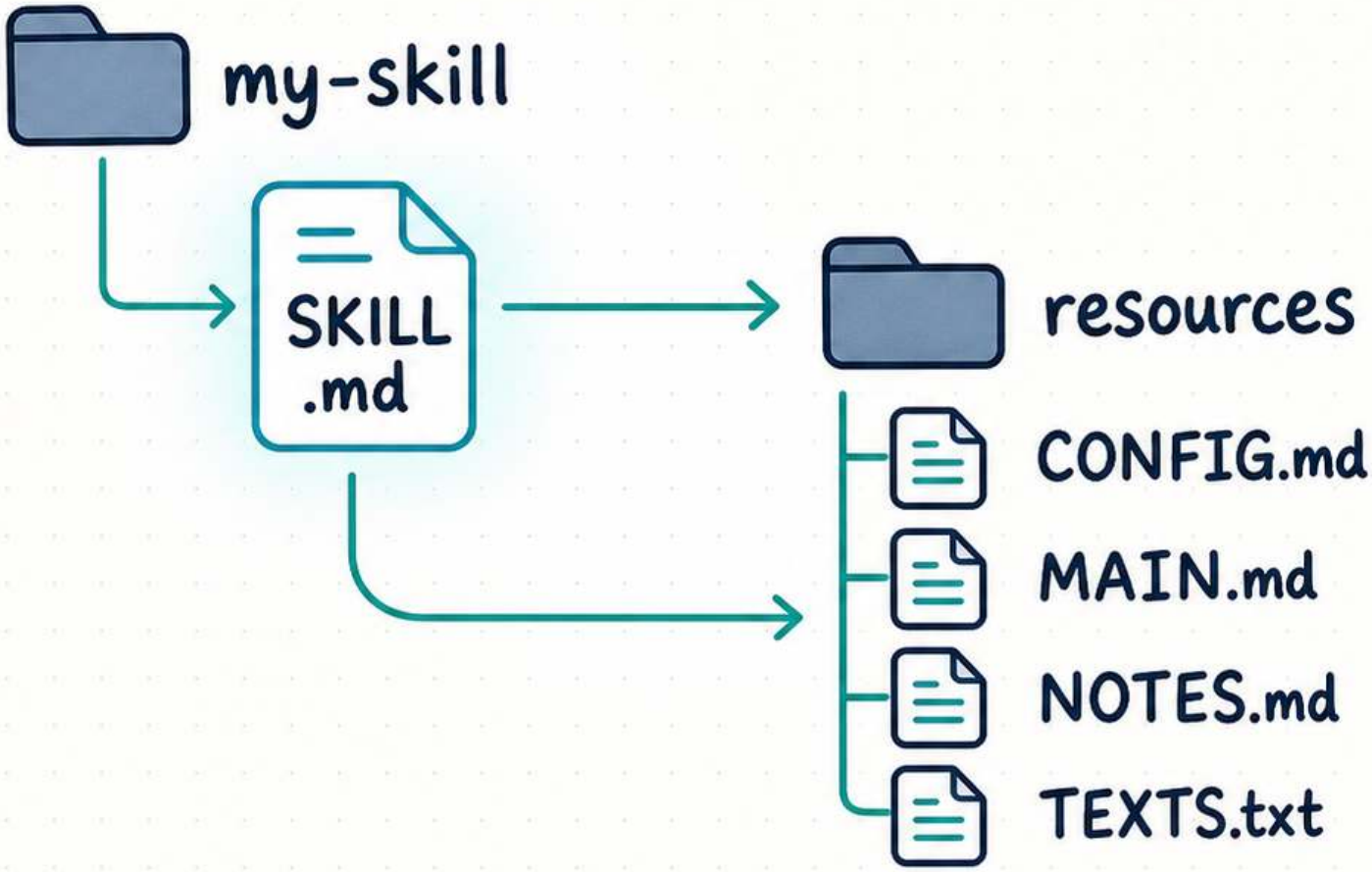
✓



Package everything into a skill

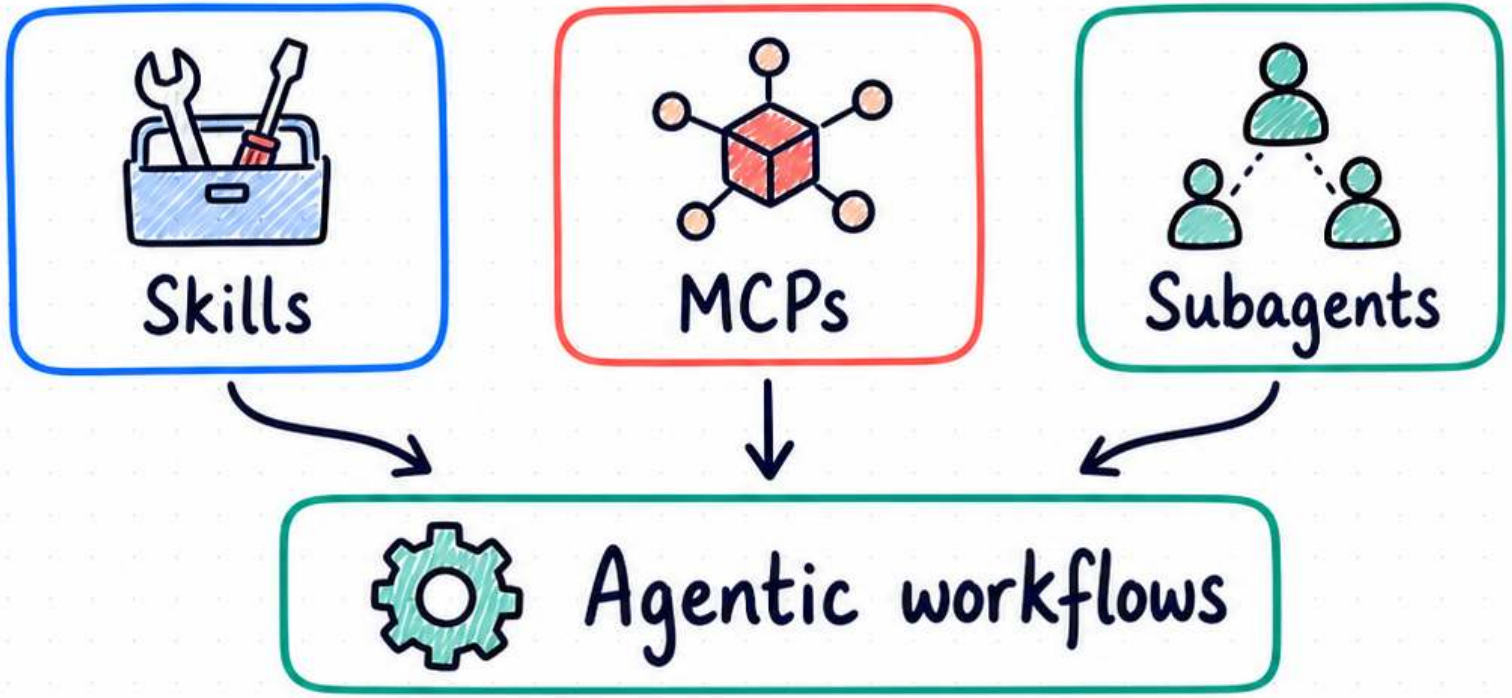


- Name
- Description
- Instructions



Building Powerful Agentic Workflows

Agents combine skills, MCPs, and subagents to create powerful agentic workflows.



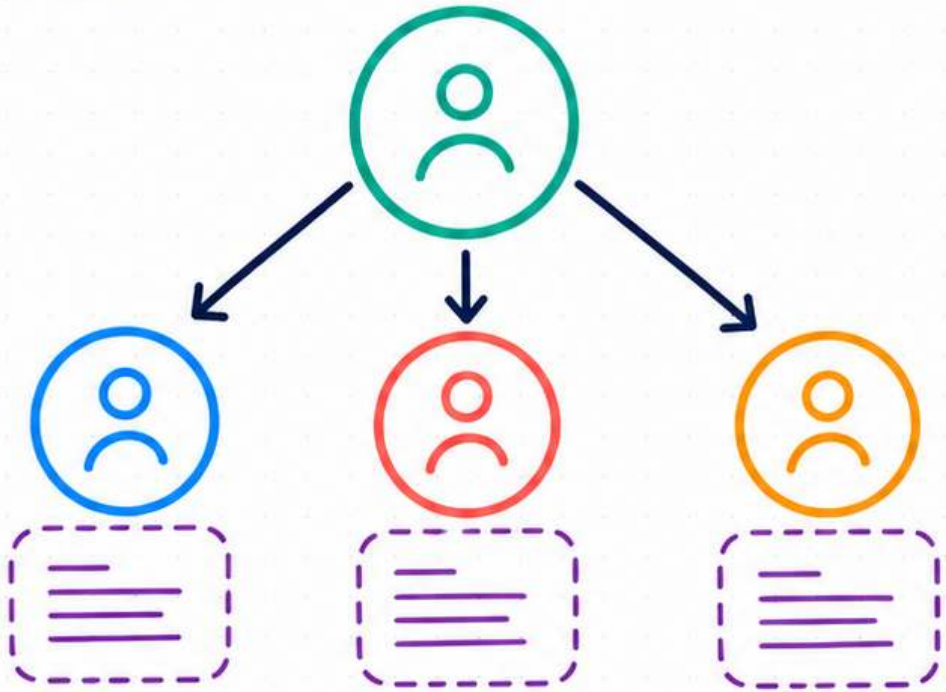
Skills make workflows deterministic because agents know which steps to take.

Subagents



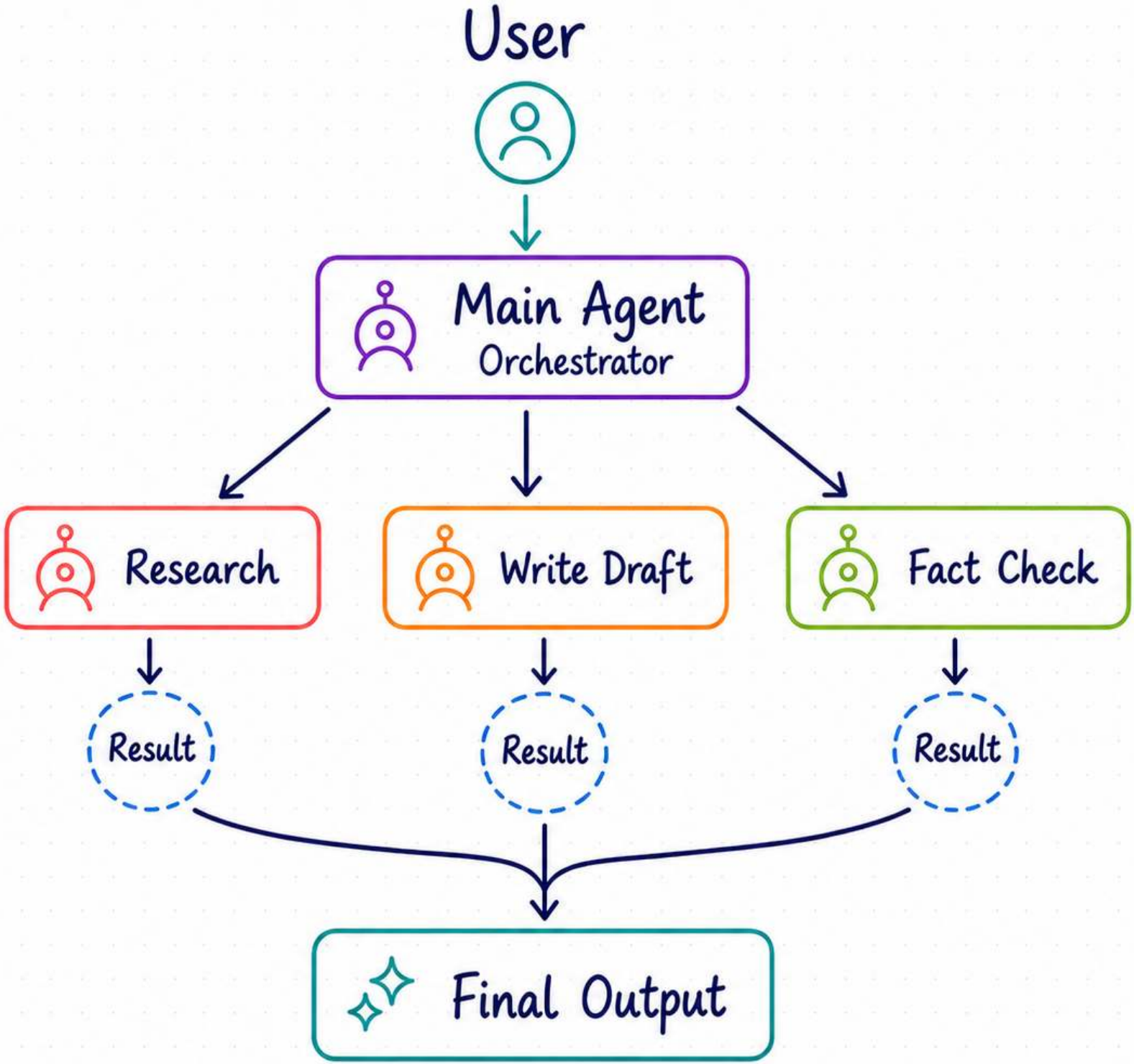
What is a Subagent?

A subagent is an agent spawned by another agent to handle a specific task.



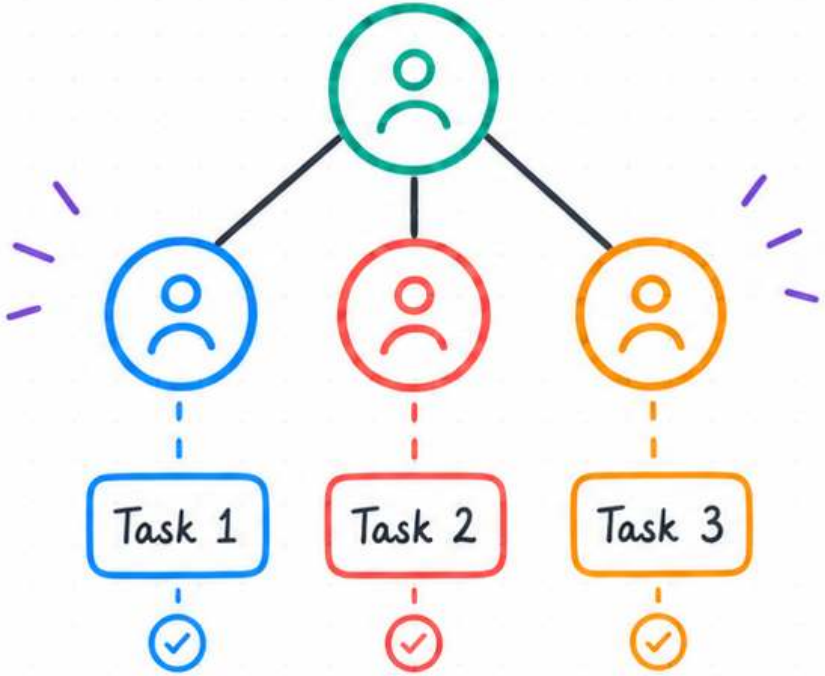


Every subagent has its own context window, independent of the main agent.


User
Main Agent
Orchestrator
Research
Write Draft
Fact Check
Result
Result
Result
Final Output

Why Subagents?

⇒ Parallelism



One main agent delegates to multiple subagents that run at the same time.

⇒ Context Management



Each subagent works with its own focused context window.